



รายงานปฏิบัติการกลุ่ม

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

กรณีศึกษา

Group 8: City Skyline

การหา Next Greater Element ด้วย Monotonic Stack

และการประยุกต์ใช้กับ Largest Rectangle in Histogram

จัดทำโดย

1. กิตติมาภรณ์ รถทอง รหัสนักศึกษา 68122250018
2. ธนารักษ์ จุลแดง รหัสนักศึกษา 68122250025
3. ภูวเดช ตรังครนสิน รหัสนักศึกษา 68122250002
4. อาลีเยห์ อาลี รหัสนักศึกษา 68122250024

เสนอ

อาจารย์กัลยณัฏฐ์ กุหลาบเพชรทอง

รายงานฉบับนี้เป็นส่วนหนึ่งของการศึกษารายวิชา CSD2103 การออกแบบและวิเคราะห์ขั้นตอนวิธี

สาขาวิชาวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏสวนสุนันทา

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

บทที่ 1

บทนำ

1. ที่มาและความสำคัญ

ในปัจจุบันการออกแบบและวิเคราะห์อัลกอริทึมมีบทบาทสำคัญในการพัฒนาโปรแกรมคอมพิวเตอร์ เนื่องจากอัลกอริทึมที่ดีสามารถช่วยลดระยะเวลาในการประมวลผลและลดการใช้ทรัพยากรของระบบได้ การเลือกใช้โครงสร้างข้อมูลที่เหมาะสมจึงเป็นส่วนสำคัญในการออกแบบอัลกอริทึม โดยเฉพาะโครงสร้างข้อมูลแบบ Stack ซึ่งมีหลักการทำงานแบบ Last In, First Out (LIFO) คือข้อมูลที่ถูกเพิ่มเข้าสู่ Stack เป็นลำดับสุดท้ายจะถูกนำออกก่อน

กรณีศึกษา City Skyline เป็นปัญหาที่เกี่ยวข้องกับการวิเคราะห์ข้อมูลความสูงของอาคารที่เรียงต่อกันเป็นลำดับ โดยมีวัตถุประสงค์เพื่อค้นหาอาคารที่มีความสูงมากกว่าถัดไปทางด้านขวาของอาคารแต่ละหลัง หรือที่เรียกว่า Next Greater Element (NGE) หากไม่พบอาคารที่มีความสูงมากกว่าทางด้านขวา จะกำหนดผลลัพธ์เป็น -1

ในการแก้ปัญหานี้สามารถออกแบบอัลกอริทึมได้หลายรูปแบบ โดยการศึกษาครั้งนี้จะออกแบบและเปรียบเทียบอัลกอริทึม 2 วิธี ได้แก่ Algorithm A: Brute Force แบบ Nested Loop ซึ่งใช้การตรวจสอบอาคารถัดไปที่ละตำแหน่ง และ Algorithm B: Monotonic Stack ซึ่งใช้โครงสร้างข้อมูล Stack เพื่อเก็บตำแหน่งของอาคารที่ยังไม่สามารถหาคำตอบได้

การเปรียบเทียบอัลกอริทึมทั้งสองวิธีจะพิจารณาจากขั้นตอนการทำงาน ความถูกต้องของผลลัพธ์ Time Complexity, Space Complexity รวมถึงประสิทธิภาพจากการทดลองกับข้อมูลที่มีขนาดแตกต่างกัน เพื่อแสดงให้เห็นถึงความแตกต่างระหว่างวิธีการแก้ปัญหาแบบ Brute Force และการประยุกต์ใช้ Monotonic Stack ในการลดจำนวนการตรวจสอบข้อมูลซ้ำ

2. Case Story

กรณีศึกษา City Skyline เป็นการจำลองสถานการณ์ที่มีอาคารหลายหลังเรียงต่อกัน โดยอาคารแต่ละหลังมีความสูงและตำแหน่งที่แตกต่างกัน

ระบบต้องการวิเคราะห์อาคารแต่ละหลังเพื่อค้นหาอาคารที่มีความสูงมากกว่าถัดไปทางด้านขวา ตัวอย่างเช่น กำหนดความสูงของอาคารเป็น [4, 5, 2, 10, 8]

เมื่อพิจารณาอาคารแต่ละหลังจากซ้ายไปขวา จะได้ผลลัพธ์ดังนี้

- อาคารสูง 4 → อาคารที่สูงกว่าถัดไปคือ 5
- อาคารสูง 5 → อาคารที่สูงกว่าถัดไปคือ 10
- อาคารสูง 2 → อาคารที่สูงกว่าถัดไปคือ 10

อาคารสูง 10 → ไม่มีอาคารที่สูงกว่าทางด้านขวา จึงได้ -1

อาคารสูง 8 → ไม่มีอาคารที่สูงกว่าทางด้านขวา จึงได้ -1

ดังนั้นผลลัพธ์ของตัวอย่างนี้คือ [5, 10, 10, -1, -1]

ปัญหานี้สามารถแก้ไขด้วยวิธี Brute Force ซึ่งตรวจสอบอาคารทางด้านขวาทีละหลัง หรือใช้ Monotonic Stack ซึ่งจัดเก็บตำแหน่งของอาคารที่ยังไม่พบอาคารที่สูงกว่า และสามารถลดจำนวนการตรวจสอบข้อมูลซ้ำได้

นอกจากนี้ แนวคิดของ Monotonic Stack ยังสามารถนำไปประยุกต์ใช้กับปัญหาอื่น เช่น Largest Rectangle in Histogram ซึ่งเป็นปัญหาที่ต้องหาพื้นที่สี่เหลี่ยมต่อเนื่องที่มีขนาดใหญ่ที่สุดจากชุดข้อมูลความสูงของอาคาร

3. วัตถุประสงค์

1. การจัดทำงานปฏิบัติการนี้มีวัตถุประสงค์ดังต่อไปนี้
2. เพื่อวิเคราะห์ Input, Output และข้อจำกัดของปัญหา City Skyline
3. เพื่อออกแบบอัลกอริทึมสำหรับแก้ปัญหา Next Greater Element จำนวน 2 วิธี
4. เพื่อศึกษาการประยุกต์ใช้โครงสร้างข้อมูล Stack ในการแก้ปัญหา
5. เพื่อเขียน Pseudocode และอธิบายขั้นตอนการทำงานของอัลกอริทึม
6. เพื่ออธิบายและพิสูจน์ความถูกต้องของอัลกอริทึม
7. เพื่อวิเคราะห์ Time Complexity และ Space Complexity ของอัลกอริทึม
8. เพื่อพัฒนาอัลกอริทึมเป็นโปรแกรมภาษา Java
9. เพื่อออกแบบ Test Cases และทดสอบกรณีปกติและกรณีขอบเขต
10. เพื่อทดลองและเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B
11. เพื่อสรุปข้อดี ข้อจำกัด และความเหมาะสมของอัลกอริทึมแต่ละวิธี

4. ขอบเขตงาน

1. การดำเนินงานของกรณีศึกษา City Skyline มีขอบเขตดังต่อไปนี้
2. ข้อมูลนำเข้าเป็นจำนวนเต็มที่แสดงถึงความสูงของอาคาร
3. อาคารแต่ละหลังมีตำแหน่งตามลำดับในชุดข้อมูล
4. ระบบต้องค้นหา Next Greater Element ของอาคารแต่ละหลัง
โดยพิจารณาเฉพาะอาคารที่อยู่ทางด้านขวา
5. หากไม่พบอาคารที่มีความสูงมากกว่าทางด้านขวา ให้แสดงผลเป็น -1
6. Algorithm A ใช้วิธี Brute Force แบบ Nested Loop และไม่ใช่ Stack
7. Algorithm B ใช้วิธี Monotonic Stack โดยใช้ Stack สำหรับเก็บ Index ของอาคาร

8. Monotonic Stack ต้องรักษาคุณสมบัติของ Stack ให้เป็นลำดับความสูงแบบลดลงตามแนวคิดของ Monotonic Decreasing Stack
9. โปรแกรมต้องสามารถรองรับกรณีความสูงของอาคารเพิ่มขึ้นต่อเนื่อง ลดลงต่อเนื่อง และมีความสูงเท่ากัน
10. โปรแกรมต้องสามารถรองรับกรณีข้อมูลว่างและข้อมูลที่มีอาคารเพียงหนึ่งหลัง
11. โปรแกรมต้องสามารถจัดการกรณีที่ไม่มีพบอาคารที่มีความสูงมากกว่าทางด้านขวา
12. โปรแกรมพัฒนาด้วยภาษา Java
13. ต้องมีการทดสอบและเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B

บทที่ 2

การวิเคราะห์ปัญหา

2. Input

ข้อมูลนำเข้าของระบบ **City Skyline** คือข้อมูลความสูงของอาคารที่เรียงต่อกันเป็นลำดับ โดยแต่ละข้อมูลแทนความสูงของอาคารหนึ่งหลัง และใช้ตำแหน่งของข้อมูลในชุดข้อมูลเป็นตำแหน่งของอาคาร กำหนดให้ข้อมูลความสูงของอาคารเป็นอาร์เรย์จำนวน n รายการ ดังนี้

Input = $[H_0, H_1, H_2, \dots, H_{n-1}]$

โดยที่

- H_i คือความสูงของอาคาร ณ ตำแหน่งที่ i
- n คือจำนวนอาคารทั้งหมด
- ค่าความสูงของอาคารเป็นจำนวนเต็ม

ตำแหน่งของอาคารเริ่มต้นที่ **Index 0** และเรียงจากซ้ายไปขวาตามลำดับของข้อมูลในอาร์เรย์

ตัวอย่าง

Input:[4, 5, 2, 10, 8]

หมายถึงมีอาคารทั้งหมด 5 หลัง โดยมีความสูงตามลำดับคือ 4, 5, 2, 10 และ 8

2. Output

ผลลัพธ์หลักที่ต้องการคืออาร์เรย์ที่แสดง **Next Greater Element (NGE)** ของอาคารแต่ละหลัง โดย NGE หมายถึงค่าความสูงของอาคารที่อยู่ทางด้านขวาซึ่งมีความสูงมากกว่าตัวอาคารปัจจุบัน และเป็นอาคารที่พบเป็นลำดับแรกจากทางด้านขวา

หากไม่มีอาคารที่อยู่ทางด้านขวาซึ่งมีความสูงมากกว่า ให้กำหนดผลลัพธ์เป็น -1

ตัวอย่าง

Input:[4, 5, 2, 10, 8]

Output:[5, 10, 10, -1, -1]

เนื่องจาก

อาคาร	ความสูง	Next Greater Element
อาคารที่ 1	4	5
อาคารที่ 2	5	10
อาคารที่ 3	2	10
อาคารที่ 4	10	-1
อาคารที่ 5	8	-1

นอกจากการหา Next Greater Element แล้ว

ระบบยังต้องสามารถนำข้อมูลความสูงของอาคารไปใช้ในการคำนวณ **Largest Rectangle in Histogram** ซึ่งเป็นพื้นที่สี่เหลี่ยมต่อเนื่องที่มีพื้นที่มากที่สุดจากความสูงของอาคารที่เรียงกัน

3. Constraints

ข้อจำกัดของปัญหามีดังต่อไปนี้

1. ข้อมูลนำเข้าต้องอยู่ในรูปแบบอาร์เรย์ของจำนวนเต็ม
2. ค่าความสูงของอาคารต้องสามารถนำมาเปรียบเทียบกันได้
3. การค้นหา Next Greater Element ต้องพิจารณาเฉพาะอาคารที่อยู่ทางด้านขวาของอาคารปัจจุบัน
4. อาคารที่อยู่ทางด้านซ้ายไม่สามารถนำมาเป็นคำตอบของอาคารปัจจุบันได้
5. หากไม่มีอาคารที่มีความสูงมากกว่าทางด้านขวา ต้องคืนค่าเป็น -1
6. ค่าความสูงที่เท่ากันไม่ถือว่าเป็น Next Greater Element
7. Algorithm A ต้องใช้วิธี **Brute Force แบบ Nested Loop** และไม่ใช่ Stack
8. Algorithm B ต้องใช้ **Monotonic Stack**
9. Algorithm B ต้องเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element
10. Algorithm B ต้องรักษาลำดับความสูงของอาคารใน Stack แบบ **Monotonic Decreasing**
11. เมื่อพบอาคารใหม่ที่มีความสูงมากกว่ายอด Stack ต้องทำการ Pop Index ออกจาก Stack และบันทึกความสูงของอาคารปัจจุบันเป็น Next Greater Element
12. ผลลัพธ์ของ Algorithm A และ Algorithm B ต้องเหมือนกันสำหรับ Input เดียวกัน
13. โปรแกรมต้องสามารถตรวจสอบ **Invalid Input** และจัดการข้อผิดพลาดจากการป้อนข้อมูลได้
14. โปรแกรมต้องสามารถรองรับกรณีข้อมูลว่างได้
15. จำนวนอาคารต้องไม่เป็นค่าติดลบ
16. โปรแกรมต้องสามารถจัดการข้อมูลจำนวนมากได้ตามขนาดการทดลองที่กำหนด

17. ข้อมูลความสูงต้องสามารถนำไปใช้คำนวณ Largest Rectangle in Histogram ได้

4. Assumptions

ในการออกแบบและพัฒนาอัลกอริทึม กำหนดข้อตกลงดังต่อไปนี้

1. ข้อมูลความสูงของอาคารเป็นจำนวนเต็ม
2. ข้อมูลอาคารถูกจัดเรียงตามตำแหน่งจากซ้ายไปขวา
3. ตำแหน่งแรกของอาคารกำหนดเป็น Index 0
4. Position Index ของอาคารตรงกับ Index ของข้อมูลในอาร์เรย์
5. Next Greater Element จะพิจารณาเฉพาะอาคารทางด้านขวา
6. หากมีอาคารหลายหลังที่มีความสูงมากกว่าตัวปัจจุบัน จะเลือกอาคารที่พบเป็นลำดับแรกจากทางด้านขวา
7. หากไม่มีอาคารที่สูงกว่า จะกำหนดผลลัพธ์เป็น -1
8. ค่าความสูงที่เท่ากันไม่ถือว่าเป็นค่าที่มากกว่า
9. Algorithm A และ Algorithm B ต้องให้ผลลัพธ์ที่ถูกต้องเหมือนกัน
10. Algorithm B ใช้ Stack สำหรับเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element
11. ในระหว่างการทำงานของ Algorithm B Stack จะรักษาลำดับความสูงแบบ Monotonic Decreasing
12. ในการทดลองจะใช้ข้อมูลที่กำหนดขึ้นเพื่อเปรียบเทียบประสิทธิภาพของอัลกอริทึม
13. การคำนวณ Largest Rectangle in Histogram จะพิจารณาความสูงของอาคารที่เรียงต่อเนื่องกัน

5. Edge Cases

ระบบต้องสามารถรองรับกรณีขอบเขตต่าง ๆ ดังต่อไปนี้

2.1 ข้อมูลว่าง กรณีที่ไม่มีข้อมูลอาคาร เช่น

Input: []

Output: []

ระบบต้องสามารถประมวลผลข้อมูลว่างได้โดยไม่เกิดข้อผิดพลาดจากการเข้าถึงข้อมูลที่ไม่มีอยู่ และไม่ควรถ่าย Pop จาก Stack ที่ว่าง

2.2 ตึกสูงขึ้นต่อเนื่อง (Increasing)

Input: [1, 2, 3, 4, 5]

Output: [2, 3, 4, 5, -1]

กรณีนี้อาคารแต่ละหลังจะพบอาคารที่สูงกว่าทางด้านขวาเป็นอาคารถัดไป

ยกเว้นอาคารสุดท้ายซึ่งไม่มีอาคารทางด้านขวา

2.3 ตีกลงต่อเนื่อง (Decreasing)

Input: [5, 4, 3, 2, 1]

Output: [-1, -1, -1, -1, -1]

กรณีนี้ไม่มีอาคารทางด้านขวาที่มีความสูงมากกว่าอาคารปัจจุบัน จึงได้ค่า -1 ทุกตำแหน่ง

2.4 ตีกลงเท่ากันทั้งหมด

Input: [5, 5, 5, 5]

Output: [-1, -1, -1, -1]

เนื่องจากค่าที่เท่ากันไม่ถือว่าเป็นค่าที่สูงกว่า

2.5 ตีกลับสูง-ต่ำ

Input: [2, 8, 3, 7, 1, 6]

Output: [8, -1, 7, -1, 6, -1]

กรณีนี้ใช้ทดสอบการทำงานของ Algorithm A และ Algorithm B

เมื่อข้อมูลมีรูปแบบความสูงสลับระหว่างอาคารสูงและอาคารต่ำ

2.6 มีตีเดียว

Input: [10]

Output: [-1]

เนื่องจากไม่มีอาคารทางด้านขวา จึงไม่สามารถหา Next Greater Element ได้

2.7 ไม่มีตีที่สูงกว่าตัวถัดไปเลย

Input: [10, 8, 6, 4, 2]

Output: [-1, -1, -1, -1, -1]

กรณีนี้ไม่มีอาคารทางด้านขวาที่มีความสูงมากกว่าอาคารปัจจุบัน จึงได้ค่า -1 ทุกตำแหน่ง

2.8 ตีสูงที่สุดอยู่ตำแหน่งแรก

Input: [10, 8, 6, 4, 2]

Output: [-1, -1, -1, -1, -1]

อาคารที่มีความสูงมากที่สุดอยู่ตำแหน่งแรก จึงไม่มีอาคารทางด้านขวาที่สูงกว่า
และอาคารที่เหลือก็ไม่มีอาคารทางด้านขวาที่สูงกว่าตนเองเช่นกัน

2.9 ตีสูงที่สุดอยู่ตำแหน่งสุดท้าย

Input: [2, 3, 4, 5, 10]

Output: [3, 4, 5, 10, -1]

กรณีนี้อาคารส่วนใหญ่สามารถพบ Next Greater Element ได้

โดยอาคารที่สูงที่สุดซึ่งอยู่ตำแหน่งสุดท้ายไม่มีอาคารทางด้านขวาที่สูงกว่า

6. กรณีปกติ

กรณีปกติคือกรณีที่ข้อมูลมีอาคารหลายหลังและมีทั้งอาคารที่สามารถค้นหา Next Greater Element ได้และอาคารที่ไม่สามารถค้นหาได้

ตัวอย่าง

Input: [4, 5, 2, 10, 8]

Output: [5, 10, 10, -1, -1]

ระบบสามารถประมวลผลข้อมูลและคืนค่าผลลัพธ์ได้อย่างถูกต้อง โดยผลลัพธ์จาก Algorithm A และ Algorithm B ต้องเหมือนกัน

7. กรณีที่อาจทำให้โปรแกรมผิดพลาด

กรณีที่อาจทำให้โปรแกรมเกิดข้อผิดพลาด ได้แก่

1. ผู้ใช้ไม่ป้อนข้อมูล
2. ผู้ใช้ป้อนข้อมูลที่ไม่ใช่จำนวนเต็ม
3. รูปแบบข้อมูลนำเข้าไม่ถูกต้อง
4. ผู้ใช้ระบุจำนวนอาคารเป็นค่าติดลบ
5. โปรแกรมพยายามเข้าถึง Index ที่อยู่นอกขอบเขตของอาร์เรย์
6. มีการใช้งาน Stack โดยไม่มีการตรวจสอบว่า Stack ว่างหรือไม่
7. มีการเรียก peek() หรือ pop() จาก Stack ที่ว่าง
8. ข้อมูลมีขนาดใหญ่มากจนใช้หน่วยความจำเกินกว่าที่ระบบรองรับ
9. การคำนวณหรือการจัดการ Index ผิดพลาดจนทำให้ผลลัพธ์ไม่ตรงกับข้อมูลนำเข้า
10. จำนวนอาคารและจำนวนข้อมูลความสูงไม่ตรงกัน

ดังนั้นโปรแกรมต้องมีการตรวจสอบ Input และจัดการ Error ที่เหมาะสมก่อนเริ่มการประมวลผล

8. Operation ที่เกิดขึ้นบ่อย

Operation หลักที่เกิดขึ้นในการแก้ปัญหา ได้แก่

1. การเปรียบเทียบความสูงของอาคาร
2. การเข้าถึงข้อมูลในอาร์เรย์
3. การตรวจสอบตำแหน่งของอาคาร
4. การตรวจสอบเงื่อนไขของ Loop
5. การ Push Index ลงใน Stack
6. การ Pop Index ออกจาก Stack
7. การตรวจสอบว่า Stack ว่างหรือไม่
8. การเข้าถึง Index บนสุดของ Stack ด้วย peek()
9. การกำหนดค่าผลลัพธ์ของ Next Greater Element
10. การตรวจสอบและจัดการ Input

สำหรับ **Algorithm B** การดำเนินการ Push และ Pop ของ Stack เป็น Operation

สำคัญที่เกิดขึ้นระหว่างการค้นหา Next Greater Element ในการทดลอง โปรแกรมจะนับจำนวน Operation

ที่สำคัญของแต่ละ Algorithm เพื่อใช้ประกอบการเปรียบเทียบประสิทธิภาพ

โดยเฉพาะจำนวนการเปรียบเทียบและการดำเนินการ Push และ Pop ของ Stack

9. Operation ที่อาจใช้เวลาามากที่สุด

สำหรับ **Algorithm A: Brute Force Operation**

ที่ใช้เวลามากที่สุดคือการเปรียบเทียบความสูงของอาคารหลายตำแหน่ง เนื่องจากสำหรับอาคารแต่ละหลัง

Algorithm จะตรวจสอบอาคารทางด้านขวาทีละตำแหน่งจนกว่าจะพบอาคารที่สูงกว่า

หรือจนกว่าจะตรวจสอบครบทุกตำแหน่ง หากอาคารที่สูงกว่าอยู่ไกลหรือไม่มีอาคารที่สูงกว่า

จะต้องทำการเปรียบเทียบหลายครั้ง ในกรณีเลวร้ายที่สุด เช่น ข้อมูลเรียงจากสูงไปต่ำ

จะต้องตรวจสอบอาคารทางด้านขวาเกือบทั้งหมดของแต่ละตำแหน่ง ทำให้มี **Time Complexity เป็น $O(n^2)$**

สำหรับ **Algorithm B: Monotonic Stack Operation** ที่สำคัญคือการเปรียบเทียบความสูง การ Push

และการ Pop Index ใน Stack แม้จะมี while loop อยู่ภายใน for loop แต่แต่ละ Index จะถูก Push

อย่างมากหนึ่งครั้งและ Pop อย่างมากหนึ่งครั้ง ตลอดการประมวลผล

ดังนั้นจำนวนการดำเนินการของ Stack โดยรวมจึงเป็นสัดส่วนกับจำนวนข้อมูล และเมื่อวิเคราะห์ด้วยแนวคิด

Amortized Analysis จะได้ **Time Complexity รวมเป็น $O(n)$**

บทที่ 3

การออกแบบอัลกอริทึม

ในการแก้ปัญหา City Skyline: การหาช่วงตึกที่มองเห็นได้ด้วย Monotonic Stack

ได้ออกแบบอัลกอริทึมสำหรับการหา Next Greater Element (NGE) จำนวน 2 วิธี ได้แก่ Algorithm A:

Brute Force แบบ Nested Loop และ Algorithm B: Monotonic Stack

โดยทั้งสองอัลกอริทึมรับข้อมูลความสูงของอาคารในรูปแบบ Array

และต้องให้ผลลัพธ์เหมือนกันสำหรับข้อมูลนำเข้าเดียวกัน

Algorithm A ใช้เป็นวิธีพื้นฐานสำหรับเปรียบเทียบประสิทธิภาพ ส่วน Algorithm B ใช้ Monotonic Stack

เพื่อลดการตรวจสอบข้อมูลซ้ำและลดเวลาในการประมวลผลเมื่อจำนวนข้อมูลมีขนาดใหญ่

1. Algorithm A: Brute Force แบบ Nested Loop

1.1 ชื่ออัลกอริทึม Brute Force Next Greater Element

1.2 Algorithm A ใช้วิธีการค้นหาแบบ **Brute Force** โดยพิจารณาอาคารแต่ละหลังจากซ้ายไปขวา และตรวจสอบอาคารที่อยู่ทางด้านขวาทีละหลังจนกว่าจะพบอาคารที่มีความสูงมากกว่าอาคารปัจจุบันเมื่อพบอาคารที่สูงกว่าจะกำหนดความสูงของอาคารนั้นเป็นคำตอบของอาคารปัจจุบันและหยุดการค้นหาด้วย BREAK หากตรวจสอบอาคารทางด้านขวาจนครบแล้วไม่พบอาคารที่สูงกว่า จะกำหนดผลลัพธ์เป็น -1

1.3 Input ข้อมูลความสูงของอาคารจำนวน n อาคาร ซึ่งเก็บอยู่ใน Array heights

โดยแต่ละตำแหน่งแทนความสูงของอาคารหนึ่งหลัง

ตัวอย่าง

Heights = [4, 5, 2, 10, 8]

1.4 Output Array ที่แสดงความสูงของอาคารที่สูงกว่าถัดไปทางด้านขวาของอาคารแต่ละหลัง

หากไม่พบอาคารที่สูงกว่าให้แสดงค่า -1

ตัวอย่าง

Input: [4, 5, 2, 10, 8]

Output:

[5, 10, 10, -1, -1]

1.5 ขั้นตอนการทำงาน

1. รับข้อมูลความสูงของอาคารใน Array heights
2. สร้าง Array result ขนาด n
3. พิจารณาอาคารแต่ละหลังตั้งแต่ Index 0 ถึง n - 1
4. กำหนดค่าเริ่มต้นของ result[i] เป็น -1
5. เริ่มค้นหาอาคารทางด้านขวา โดยเริ่มจาก $j = i + 1$
6. เปรียบเทียบ heights[j] กับ heights[i]
7. หาก heights[j] > heights[i] ให้กำหนด result[i] = heights[j]
8. หยุดการค้นหาของอาคารปัจจุบันด้วย BREAK
9. หากไม่พบอาคารที่สูงกว่า ให้คงค่า result[i] เป็น -1
10. ทำซ้ำจนกว่าจะตรวจสอบอาคารครบทุกหลัง
11. คืนค่า result

1.6 ข้อดี

1. มีแนวคิดและขั้นตอนการทำงานที่เข้าใจง่าย
2. สามารถตรวจสอบความถูกต้องของผลลัพธ์ได้ง่าย
3. ไม่จำเป็นต้องใช้ Stack
4. สามารถนำไปพัฒนาเป็นโปรแกรมได้ง่าย
5. เหมาะสำหรับใช้เป็นวิธีพื้นฐานในการเปรียบเทียบประสิทธิภาพกับ Algorithm ที่ใช้ Monotonic Stack

1.7 ข้อจำกัด

1. อาจต้องตรวจสอบอาคารทางด้านขวาหลายตำแหน่งสำหรับอาคารแต่ละหลัง
2. มีการเข้าถึงข้อมูลซ้ำหลายครั้ง
3. เมื่อจำนวนอาคารเพิ่มขึ้น เวลาที่ใช้ในการประมวลผลจะเพิ่มขึ้น
4. ไม่เหมาะกับข้อมูลจำนวนมากเมื่อเปรียบเทียบกับ Monotonic Stack
5. ในกรณีเลวร้ายที่สุดมี Time Complexity เป็น $O(n^2)$

2. Algorithm B: Monotonic Stack

2.1 ชื่ออัลกอริทึม Monotonic Stack Next Greater Element

2.2 แนวคิดหลัก Algorithm B ใช้โครงสร้างข้อมูล **Stack** สำหรับเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element ทางด้านขวา โดยรักษาลำดับความสูงของอาคารใน Stack ให้เป็น Monotonic Increasing เมื่อพิจารณาอาคารปัจจุบัน หากอาคารปัจจุบันมีความสูงมากกว่าอาคารที่อยู่บนยอด Stack จะทำการ Pop Index ของอาคารดังกล่าวออกจาก Stack และกำหนดอาคารปัจจุบันเป็น Next Greater Element จากนั้นตรวจสอบต่อไปจนกว่า Stack จะว่างหรืออาคารปัจจุบันไม่สูงกว่าอาคารบนยอด Stack แล้วจึง Push Index ของอาคารปัจจุบันเข้า Stack หลักการนี้ช่วยลดการตรวจสอบข้อมูลซ้ำ โดยแต่ละ Index จะถูก Push และ Pop อย่างมากหนึ่งครั้งตลอดการทำงาน

2.3 Input ข้อมูลความสูงของอาคารจำนวน n อาคาร ซึ่งเก็บอยู่ใน Array heights

ตัวอย่าง

Heights = [4, 5, 2, 10, 8]

2.4 Output Array ที่เก็บ Next Greater Element ของอาคารแต่ละหลัง

หากไม่พบอาคารที่สูงกว่าทางด้านขวาให้กำหนดค่าเป็น -1

Output = [5, 10, 10, -1, -1]

2.5 ขั้นตอนการทำงาน

1. รับข้อมูล heights
2. สร้าง Array result ขนาด n
3. กำหนดค่าทุกตำแหน่งของ result เป็น -1
4. สร้าง Stack ว่าง
5. เริ่มพิจารณาอาคารตั้งแต่ Index 0
6. ตรวจสอบว่า Stack ไม่ว่างและอาคารปัจจุบันสูงกว่าอาคารบนยอด Stack หรือไม่
7. หากเป็นจริง ให้ Pop Index ออกจาก Stack
8. กำหนด result[index] = heights[i]
9. ทำซ้ำข้อ 6-8 จนกว่าเงื่อนไขจะเป็นเท็จ

10. Push Index ปัจจุบันเข้า Stack
11. ไปยังอาคารถัดไป
12. เมื่อประมวลผลครบทุกอาคาร อาคารที่ยังเหลือใน Stack จะไม่มีอาคารที่สูงกว่าทางด้านขวา จึงมีค่า 1
13. คืนค่า result

2.6 Invariant ของ Monotonic stack ระหว่างการทำงานของ Algorithm B Stack จะเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element และค่าความสูงของอาคารที่อยู่ใน Stack จะเรียงจากน้อยไปมากตามลำดับจากล่างขึ้นบน เมื่อพบอาคารใหม่ที่สูงกว่ายอด Stack อาคารดังกล่าวสามารถเป็น Next Greater Element ของ Index ที่ถูก Pop ออกได้ ดังนั้น Stack จึงรักษาคุณสมบัติ Monotonic Increasing ตลอดการทำงาน

2.7 ข้อดี

1. มีประสิทธิภาพสูงกว่า Brute Force สำหรับข้อมูลจำนวนมาก
2. ลดการตรวจสอบข้อมูลซ้ำ
3. อาคารแต่ละตำแหน่งถูก Push และ Pop อย่างมากหนึ่งครั้ง
4. มี Time Complexity โดยรวมเป็น $O(n)$
5. สามารถประยุกต์ใช้กับปัญหาอื่น เช่น **Largest Rectangle in Histogram** ได้

2.8 ข้อจำกัด

1. มีแนวคิดที่ซับซ้อนกว่า Brute Force
2. ต้องเข้าใจหลักการทำงานของ Monotonic Stack
3. ต้องใช้พื้นที่เพิ่มเติมสำหรับ Stack
4. การแก้ไขหรือประยุกต์ใช้ต้องระมัดระวังเรื่องเงื่อนไขการ Pop

2.9 Time Complexity

1. **Best Case:** $O(n)$
2. **Average Case:** $O(n)$
3. **Worst Case:** $O(n)$

แม้ Algorithm B จะมี Loop WHILE อยู่ภายใน Loop FOR แต่ไม่ได้ทำให้มี Time Complexity เป็น $O(n^2)$ เพราะแต่ละ Index สามารถถูก Push เข้า Stack ได้อย่างมากหนึ่งครั้ง และถูก Pop ออกจาก Stack

ได้อย่างมากหนึ่งครั้งตลอดการทำงาน ดังนั้นจำนวน Operation รวมจึงเป็นสัดส่วนกับ n และมี Time Complexity เป็น $O(n)$

2.10 Space Complexity

ในกรณีเลวร้ายที่สุด Stack อาจเก็บ Index ของอาคารทั้งหมดจำนวน n ตำแหน่ง เช่น กรณีอาคารเรียงจากสูงไปต่ำ ดังนั้น Auxiliary Space Complexity ของ Algorithm B เท่ากับ $O(n)$

3. Pseudocode

ในการออกแบบอัลกอริทึมสำหรับปัญหา Next Greater Element ได้กำหนด Pseudocode ของ Algorithm A และ Algorithm B เพื่อแสดงขั้นตอนการทำงานอย่างละเอียด และสามารถนำไปพัฒนาเป็นโปรแกรมภาษา Java ได้

3.1 Pseudocode ของ Algorithm A: Brute Force

ALGORITHM BruteForceNGE(heights)

INPUT: heights คือ Array ที่เก็บความสูงของอาคารจำนวน n อาคาร

OUTPUT: result คือ Array ที่เก็บความสูงของอาคารที่สูงกว่าถัดไป ทางด้านขวาของแต่ละอาคาร หากไม่พบให้เป็น -1

```
1. Create result array with size n
2. FOR i ← 0 TO n - 1
3.   result[i] ← -1
4.   FOR j ← i + 1 TO n - 1
5.     IF heights[j] > heights[i]
6.       result[i] ← heights[j]
7.       BREAK
8.   END IF
9. END FOR
10. END FOR
11. RETURN result
```

Pseudocode นี้สอดคล้องกับหลักการ Brute Force ที่ใช้ Loop ชั้นนอกสำหรับอาคารแต่ละหลัง และ Loop ชั้นในสำหรับค้นหาอาคารที่อยู่ทางด้านขวา

3.2 Pseudocode ของ Algorithm B: Monotonic Stack

ALGORITHM MonotonicStackNGE(heights)

INPUT: heights คือ Array ที่เก็บความสูงของอาคารจำนวน n อาคาร

OUTPUT: result คือ Array ที่เก็บความสูงของอาคารที่สูงกว่าถัดไป

ทางด้านขวาของแต่ละอาคาร หากไม่พบให้เป็น -1

1. Create result array with size n
2. Fill all elements in result with -1
3. Create an empty Stack
4. FOR $i \leftarrow 0$ TO $n - 1$
5. WHILE Stack is not empty AND
 heights[i] > heights[Stack.peek()]
6. index $\leftarrow$ Stack.pop()
7. result[index] $\leftarrow$ heights[i]
8. END WHILE
9. Stack.push(i)
10. END FOR
11. RETURN result

ALGORITHM MonotonicStackNGE(heights)

INPUT: heights คือ Array ที่เก็บความสูงของอาคารจำนวน n อาคาร

OUTPUT: result คือ Array

ที่เก็บความสูงของอาคารที่สูงกว่าถัดไปทางด้านขวาของแต่ละอาคารหากไม่พบให้เป็น -1

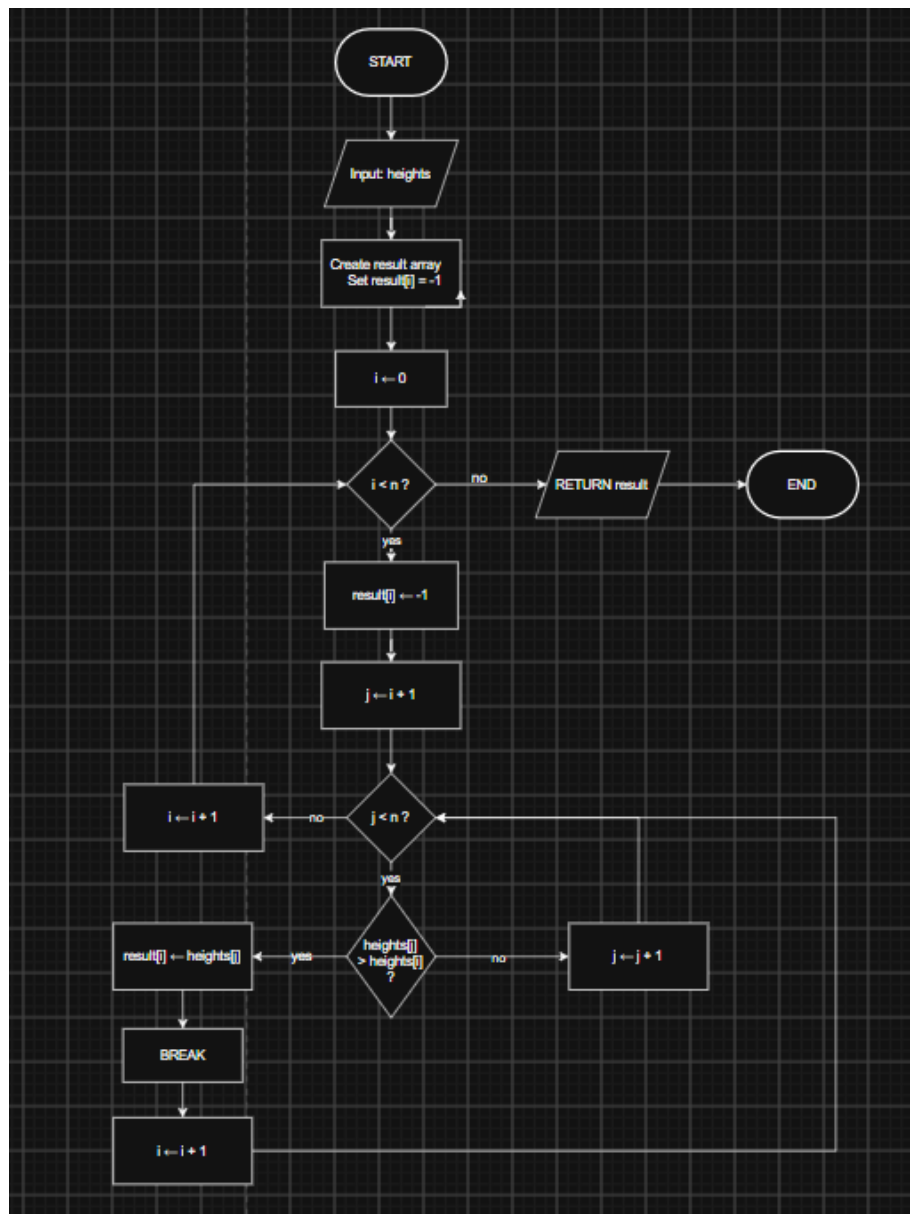
1. Create result array with size n
2. Fill all elements in result with -1
3. Create an empty Stack
4. FOR $i \leftarrow 0$ TO $n - 1$
5. WHILE Stack is not empty AND
 $\text{heights}[i] > \text{heights}[\text{Stack.peek}()]$
6. $\text{index} \leftarrow \text{Stack.pop}()$
7. $\text{result}[\text{index}] \leftarrow \text{heights}[i]$
8. END WHILE
9. $\text{Stack.push}(i)$
10. END FOR
11. RETURN result

Pseudocode นี้ตรงกับ Algorithm B ในงานเดิม โดยใช้ Stack.peek() เพื่อตรวจสอบยอด Stack, Stack.pop() เพื่อเอา Index ที่พบคำตอบออก และ Stack.push(i) เพื่อเก็บ Index ของอาคารปัจจุบัน

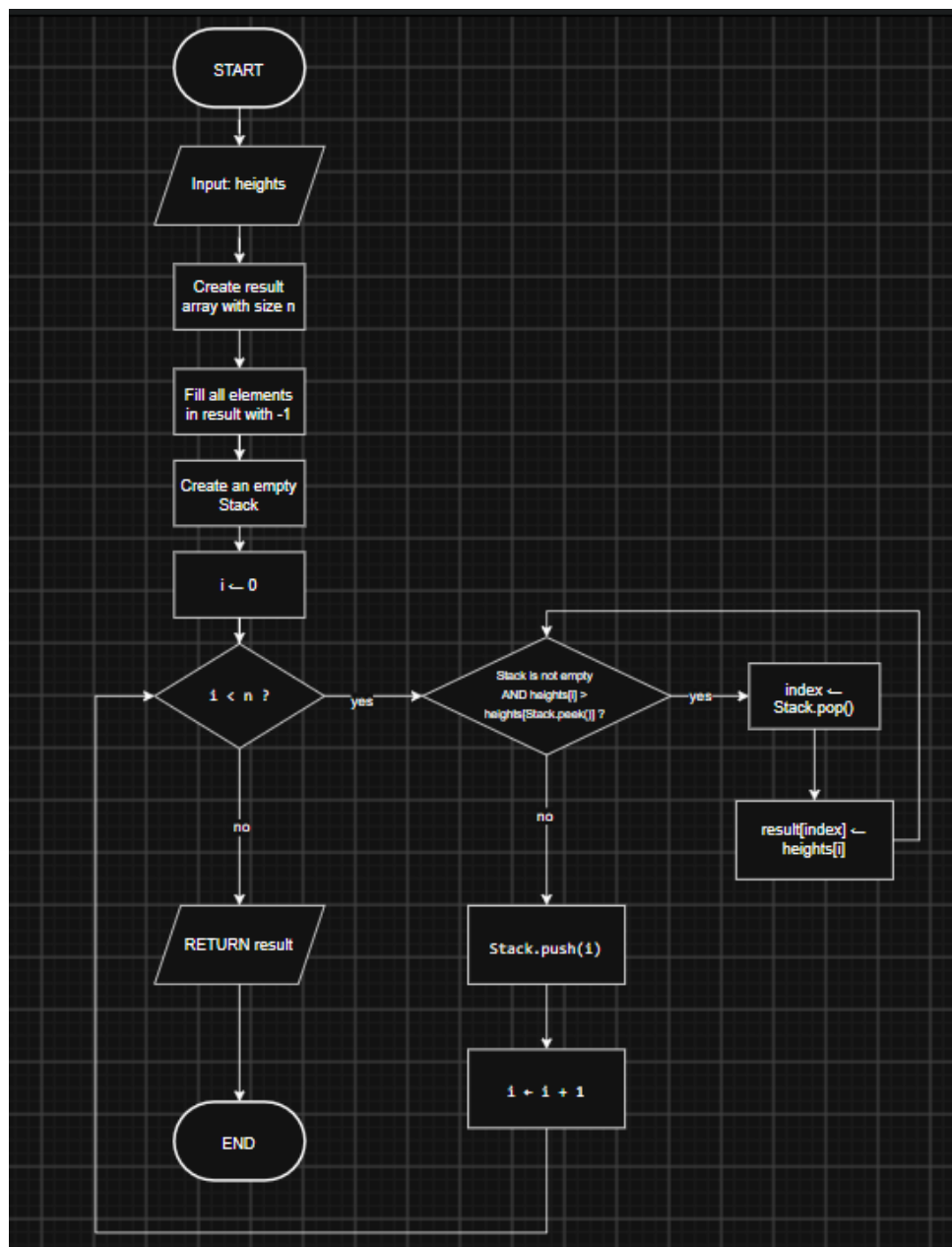
4. Flowchart

Flowchart ใช้สำหรับแสดงลำดับขั้นตอนการทำงานของ Algorithm A และ Algorithm B ในรูปแบบแผนผัง เพื่อให้สามารถมองเห็นลำดับการทำงาน การตัดสินใจ และการวนซ้ำของอัลกอริทึม

4.1 Flowchart ของ Algorithm A: Brute Force



4.2 Flowchart ของ Algorithm B: Monotonic Stack



5. ตัวอย่างการทำงานแบบ Step-by-step

เพื่อให้สามารถเปรียบเทียบ Algorithm A และ Algorithm B ได้อย่างชัดเจน

จะใช้ข้อมูลชุดเดียวกันในการสาธิตการทำงาน Heights = [4, 5, 2, 10, 8]

ผลลัพธ์ที่ถูกต้องคือ Result = [5, 10, 10, -1, -1]

การใช้ข้อมูลชุดเดียวกันสำหรับทั้งสองอัลกอริทึมสอดคล้องกับตัวอย่างในเอกสารเดิมของงาน

5.1 ตัวอย่างการทำงานของ Algorithm A: Brute Force

อาคาร	Height	การตรวจสอบทางขวา	NGE
Index 0	4	เปรียบเทียบ 5 $\rightarrow$ พบว่า $5 > 4$	5
Index 1	5	2 ไม่มากกว่า 5 $\rightarrow 10 > 5$	10
Index 2	2	$10 > 2$	10
Index 3	10	8 ไม่มากกว่า 10 และไม่มีข้อมูลต่อ	-1
Index 4	8	ไม่มีอาคารทางขวา	-1

ดังนั้น Result = [5, 10, 10, -1, -1]

5.2 ตัวอย่างการทำงานของ Algorithm B: Monotonic Stack

กำหนด

Heights = [4, 5, 2, 10, 8]

Initial Result = [-1, -1, -1, -1, -1]

Initial Stack = []

Step 1 — Height = 4

Index 0

Stack ว่าง จึงทำ Push(0)

ได้ Stack = [0]

โดย Index 0 มี Height = 4

Step 2 — Height = 5

Index 1

ตรวจสอบ $5 > 4$

เป็นจริง จึง Pop(0) result[0] = 5

จากนั้น Push(1) ได้ Stack = [1]

Step 3 — Height = 2

Index 2

ตรวจสอบ $2 > 5$ เป็นเท็จ จึงไม่ Pop และทำ Push(2) ได้ Stack = [1, 2]

Step 4 — Height = 10

Index 3 ตรวจสอบกับยอด Stack: $10 > 2$ เป็นจริง

Pop(2) result[2] = 10

ตรวจสอบต่อ $10 > 5$ เป็นจริง

Pop(1) result[1] = 10

จากนั้น Push(3) ได้ Stack = [3]

Step 5 — Height = 8

Index 4 ตรวจสอบ $8 > 10$ เป็นเท็จ

จึงทำ Push(4) ได้ Stack = [3, 4]

เมื่อสิ้นสุดการทำงาน อาคารที่ยังอยู่ใน Stack ไม่มีอาคารที่สูงกว่าทางด้านขวา ดังนั้นผลลัพธ์ของ

Index เหล่านั้นจึงเป็น -1 Result = [5, 10, 10, -1, -1]

5.3 เปรียบเทียบผลลัพธ์

Algorithm	Input	Output
Algorithm A: Brute Force	[4,5,2,10,8]	[5,10,10,-1,-1]
Algorithm B: Monotonic Stack	[4,5,2,10,8]	[5,10,10,-1,-1]

จากการทดลองตัวอย่าง ทั้ง Algorithm A และ Algorithm B ให้ผลลัพธ์เหมือนกัน คือ [5, 10, 10, -1, -1] แสดงให้เห็นว่าทั้งสองอัลกอริทึมสามารถแก้ปัญหา Next Greater Element ได้อย่างถูกต้อง แม้ว่าจะใช้วิธีการทำงานแตกต่างกัน

6. ภาพสถานะ Stack

ส่วนนี้แสดงสถานะของ Stack ระหว่างการทำงานของ Algorithm B: Monotonic Stack เพื่อให้ให้เห็นการเปลี่ยนแปลงของข้อมูลภายใน Stack ในแต่ละขั้นตอน โดย Stack จะเก็บ **Index** ของอาคารที่ยังไม่พบอาคารที่มีความสูงมากกว่าทางด้านขวา

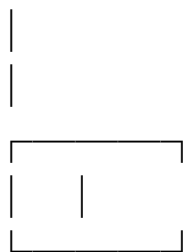
กำหนด

Heights = [4, 5, 2, 10, 8]

6.1 Initial Stack

เริ่มต้น Stack ว่าง

TOP



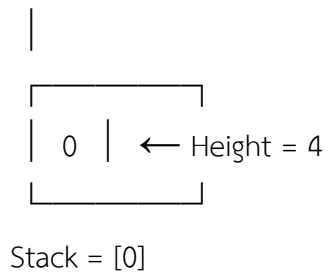
EMPTY

6.2 Step 1 — อ่านอาคารความสูง 4

i = 0

Stack ว่าง จึงทำ Push(0)

TOP

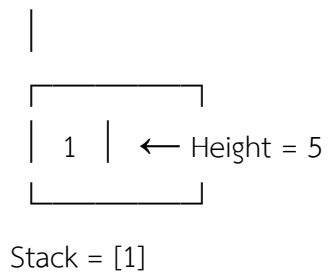


6.3 Step 2 — อ่านอาคารความสูง 5

$i = 1$ เปรียบเทียบ $5 > 4$ เป็นจริง

จึง Pop(0) result[0] = 5 จากนั้น Push(1)

TOP

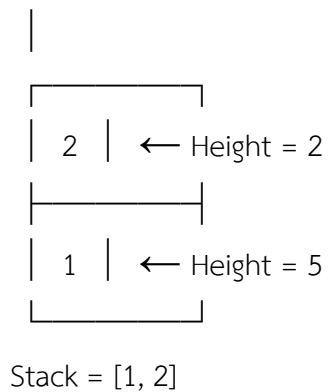


6.4 Step 3 — อ่านอาคารความสูง 2

$i = 2$ เปรียบเทียบ $2 > 5$

ไม่เป็นจริง จึงไม่ Pop ทำ Push(2)

TOP



6.5 Step 4 — อ่านอาคารความสูง 10

$i = 3$ เปรียบเทียบกับยอด Stack $10 > 2$

เป็นจริง Pop(2) result[2] = 10

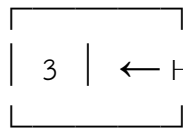
ตรวจสอบต่อ $10 > 5$ เป็นจริง

Pop(1) result[1] = 10

จากนั้น Push(3) สถานะ Stack:

TOP

|



Stack = [3]

6.6 Step 5 — อ่านอาคารความสูง 8

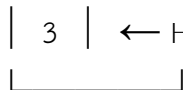
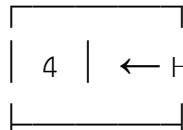
$i = 4$ เปรียบเทียบ $8 > 10$ ไม่เป็นจริง

จึงไม่ Pop และทำPush(4)

สถานะสุดท้าย

TOP

|



Stack = [3, 4]

เมื่อสิ้นสุดการทำงาน Index ที่เหลืออยู่ใน Stack คือ Index 3 และ 4

ซึ่งไม่มีอาคารที่สูงกว่าทางด้านขวา จึงมีค่าใน result เป็น -1

บทที่ 4

การอธิบายความถูกต้อง

บทนี้อธิบายเงื่อนไขและเหตุผลที่ทำให้ Algorithm A: Brute Force และ Algorithm B: Monotonic Stack สามารถแก้ปัญหา Next Greater Element ได้อย่างถูกต้อง โดยพิจารณา Precondition, Invariant, Postcondition และ Termination ของอัลกอริทึม

1. Precondition

Precondition คือเงื่อนไขที่ต้องเป็นจริงก่อนเริ่มการทำงานของอัลกอริทึม

สำหรับ Algorithm A และ Algorithm B กำหนด Precondition ดังนี้

1. ข้อมูลนำเข้า heights อยู่ในรูปแบบ Array ของจำนวนเต็ม
 2. จำนวนข้อมูลใน Array กำหนดเป็น n
 3. ตำแหน่งของอาคารเริ่มต้นที่ Index 0 และสิ้นสุดที่ Index $n - 1$
 4. ข้อมูลแต่ละตำแหน่งแทนความสูงของอาคารหนึ่งหลัง
 5. การหา Next Greater Element จะพิจารณาเฉพาะอาคารที่อยู่ทางด้านขวาของอาคารปัจจุบัน
 6. อาคารที่มีความสูงเท่ากันจะไม่ถือว่าเป็น Next Greater Element เนื่องจากต้องมีความสูง **มากกว่า** อาคารปัจจุบัน
 7. หากไม่มีอาคารที่สูงกว่าทางด้านขวา ผลลัพธ์ของตำแหน่งนั้นต้องเป็น -1
 8. สำหรับ Algorithm B ต้องสร้าง Stack ให้อยู่ในสถานะว่างก่อนเริ่มการประมวลผล
- ตัวอย่างข้อมูลนำเข้า
- heights = [4, 5, 2, 10, 8]
- ข้อมูลดังกล่าวสามารถนำไปประมวลผลด้วยทั้ง Algorithm A และ Algorithm B ได้

2. Invariant

invariant คือเงื่อนไขที่ยังคงเป็นจริงระหว่างการทำงานของ Loop หรือในแต่ละขั้นตอนของอัลกอริทึม

2.1 Invariant ของ Algorithm A: Brute Force

ในระหว่างการทำงานของ Loop ชั้นนอก เมื่อกำลังพิจารณาอาคารที่ Index i

อาคารก่อนหน้าในตำแหน่ง 0 ถึง $i - 1$ ได้ถูกประมวลผลเรียบร้อยแล้ว

สำหรับแต่ละตำแหน่งที่ประมวลผลแล้ว ค่าใน result จะเป็น Next Greater Element ที่ถูกต้องของอาคารตำแหน่งนั้น หรือเป็น -1 หากไม่พบอาคารที่สูงกว่าทางด้านขวา

สำหรับ Loop ชั้นใน จะตรวจสอบอาคารทางด้านขวาของอาคารปัจจุบันตั้งแต่ตำแหน่ง $i + 1$ เป็นต้นไปตามลำดับจากซ้ายไปขวา เมื่อพบอาคารแรกที่มีความสูงมากกว่า จะหยุดการค้นหาและบันทึกค่าดังกล่าวเป็นคำตอบ ดังนั้นอัลกอริทึมจะไม่เลือกอาคารที่สูงกว่าแต่ไม่ได้เป็นอาคารแรกทางด้านขวา

2.2 Invariant ของ Algorithm B: Monotonic Stack

ระหว่างการทำงานของ Algorithm B Stack จะเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element ทางด้านขวา

นอกจากนี้ ความสูงของอาคารที่อยู่ใน Stack จะมีลำดับแบบ Monotonic Increasing จากล่างขึ้นบน ตัวอย่างเช่น หาก Stack มี Index

[1, 2] และ $\text{heights}[1] = 5$ $\text{heights}[2] = 2$

จะเห็นว่าตามการ Push จริงในตัวอย่างนี้ค่าจากล่างขึ้นบนเป็น 5, 2 ซึ่งเป็น Monotonic Decreasing หากมองจากล่างขึ้นบน

ดังนั้นเพื่อให้สอดคล้องกับ Pseudocode ที่เราใช้จริง ต้องระบุอย่างแม่นยำว่า:

Stack จะรักษาลำดับความสูงจากมากไปน้อยเมื่ออ่านจากล่างขึ้นบน หรือจากน้อยไปมากเมื่ออ่านจากยอด Stack ลงด้านล่าง

เมื่อมีอาคารใหม่ที่สูงกว่ายอด Stack จะทำการ Pop อาคารที่เตี้ยกว่าออก และอาคารใหม่จะเป็น Next Greater Element ของอาคารเหล่านั้น

ตัวอย่าง

Heights = [4, 5, 2, 10, 8]

เมื่อถึง 10

Stack ก่อนประมวลผล 10 = [1, 2]

Height = [5, 2]

เนื่องจาก

$10 > 2$

จึง Pop Index 2 และกำหนด

$\text{result}[2] = 10$

จากนั้นตรวจสอบต่อ

$10 > 5$

จึง Pop Index 1 และกำหนด

$\text{result}[1] = 10$

จากนั้น Push Index 3 เข้า Stack

3. Postcondition

Postcondition คือเงื่อนไขที่ต้องเป็นจริงหลังจากอัลกอริทึมทำงานเสร็จสิ้น

เมื่อ Algorithm A และ Algorithm B ประมวลผลเสร็จ จะต้องมียุทธศาสตร์ดังต่อไปนี้

1. result มีจำนวนสมาชิกเท่ากับจำนวนอาคารใน heights
2. สำหรับทุก Index i หากมีอาคารทางด้านขวาที่มีความสูงมากกว่า heights[i] ผลลัพธ์ result[i] ต้องเป็นอาคารแรกที่พบจากทางด้านขวา
3. หากไม่มีอาคารที่มีความสูงมากกว่า heights[i] ทางด้านขวา result[i] ต้องมีค่าเป็น -1
4. อาคารที่มีความสูงเท่ากันจะไม่ถูกนำมาเป็น Next Greater Element
5. Algorithm A และ Algorithm B ต้องให้ผลลัพธ์เหมือนกันเมื่อรับ Input เดียวกัน

ตัวอย่าง

Input:

[4, 5, 2, 10, 8]

Output:

[5, 10, 10, -1, -1]

ตรวจสอบได้ว่า

4 → 5

5 → 10

2 → 10

10 → ไม่มี → -1

8 → ไม่มี → -1

ดังนั้นผลลัพธ์ถูกต้องตามนิยามของ Next Greater Element

4. Termination

Termination คือเงื่อนไขที่ทำให้อัลกอริทึมสามารถสิ้นสุดการทำงานได้ และไม่เกิดการวนซ้ำอย่างไม่มีที่สิ้นสุด

4.1 Termination ของ Algorithm A

Algorithm A มี Loop ชั้นนอกที่ใช้ Index i โดยเริ่มจาก 0 และเพิ่มค่าขึ้นทีละ 1 จนถึง $n - 1$

$i = 0$

$i = 1$

$i = 2$

...

$i = n - 1$

ส่วน Loop ชั้นในใช้ Index j ซึ่งเริ่มจาก $i + 1$ และเพิ่มขึ้นทีละ 1 จนถึง $n - 1$
หรือหยุดก่อนหากพบอาคารที่สูงกว่า
ดังนั้นทั้ง Loop ชั้นนอกและ Loop ชั้นในมีขอบเขตที่ชัดเจน และสามารถสิ้นสุดการทำงานได้

4.2 Termination ของ Algorithm B

Algorithm B มี Loop FOR ที่เพิ่มค่า Index i ตั้งแต่ 0 ถึง $n - 1$ ดังนั้น Loop หลักจะสิ้นสุดเมื่อประมวลผลอาคารครบทุกหลัง
ส่วน Loop WHILE จะทำงานเฉพาะเมื่อ Stack ไม่ว่างและอาคารปัจจุบันสูงกว่ายอด Stack
ทุกครั้งที่ Loop WHILE ทำงาน จะมีการ Pop Index ออกจาก Stack อย่างน้อยหนึ่งตำแหน่ง
เนื่องจากแต่ละ Index ถูก Push เข้า Stack อย่างมากหนึ่งครั้ง และถูก Pop อย่างมากหนึ่งครั้ง
จึงมีจำนวนการ Pop จำกัด และ Loop WHILE ไม่สามารถทำงานไปเรื่อย ๆ ได้อย่างไม่มีที่สิ้นสุด
ดังนั้น Algorithm B จึงสามารถสิ้นสุดการทำงานได้อย่างแน่นอน

บทที่ 5

การวิเคราะห์ประสิทธิภาพ

บทนี้เป็นการวิเคราะห์ประสิทธิภาพของ Algorithm A: Brute Force และ Algorithm B: Monotonic Stack สำหรับการแก้ปัญหา Next Greater Element โดยพิจารณา Best Case, Average Case, Worst Case, Time Complexity และ Space Complexity รวมถึงเปรียบเทียบประสิทธิภาพของอัลกอริทึมทั้งสองวิธี

1 Best Case

1.1 Algorithm A: Brute Force

Best Case ของ Algorithm A

เกิดขึ้นเมื่ออาคารที่อยู่ถัดไปทางด้านขวามีความสูงมากกว่าอาคารปัจจุบันทันที

ทำให้สามารถหยุดการค้นหาได้ตั้งแต่การเปรียบเทียบครั้งแรก

ตัวอย่างเช่น

Input = [1, 2, 3, 4, 5]

สำหรับอาคารแต่ละหลังที่ไม่ใช่อาคารสุดท้าย สามารถพบ Next Greater Element จากอาคารถัดไปได้ทันที

ดังนั้นจำนวนการตรวจสอบในแต่ละตำแหน่งมีน้อย และเวลาในการทำงานอยู่ในระดับ

$O(n)$

อย่างไรก็ตาม อาคารสุดท้ายยังต้องตรวจสอบว่าไม่มีข้อมูลทางด้านขวา

จึงไม่ทำให้ลำดับความซับซ้อนสูงกว่า $O(n)$

1.2 Algorithm B: Monotonic Stack

Best Case ของ Algorithm B คือกรณีที่สามารถประมวลผลข้อมูลโดยแต่ละ Index ถูก Push และ Pop อย่างจำกัด โดยทุก Index ถูก Push อย่างมากหนึ่งครั้ง และ Pop อย่างมากหนึ่งครั้ง

ตัวอย่างเช่น

Input = [1, 2, 3, 4, 5]

เมื่อพบอาคารที่สูงขึ้น อาจเกิดการ Pop อาคารก่อนหน้า แต่ Index ที่ถูก Pop แล้วจะไม่ถูกนำกลับเข้า Stack อีก

ดังนั้นจำนวน Operation รวมยังเป็นสัดส่วนกับจำนวนข้อมูล

Time Complexity = $O(n)$

2. Average Case

2.1 Algorithm A: Brute Force

Average Case ของ Algorithm A เกิดขึ้นเมื่อบางอาคารสามารถพบ Next Greater Element ได้เร็ว

ขณะที่บางอาคารต้องตรวจสอบอาคารทางด้านขวาหลายตำแหน่งก่อนพบคำตอบ

จำนวนการเปรียบเทียบจึงขึ้นอยู่กับรูปแบบของข้อมูล

และในภาพรวมมีการตรวจสอบข้อมูลซ้ำจากหลายตำแหน่ง

ดังนั้น Average Case ของ Algorithm A มี Time Complexity เป็น $O(n^2)$

2.2 Algorithm B: Monotonic Stack

Average Case ของ Algorithm B ยังคงเป็น $O(n)$ เนื่องจากแม้จะมีการวน WHILE ภายใน FOR แต่ละ Index มีจำนวนการดำเนินการกับ Stack จำกัด แต่ละ Index สามารถ

- Push ได้อย่างมาก 1 ครั้ง
- Pop ได้อย่างมาก 1 ครั้ง

ดังนั้นจำนวน Operation รวมตลอดการทำงานเป็นสัดส่วนกับ n

Time Complexity = $O(n)$

3 Worst Case

3.1 Algorithm A: Brute Force

Worst Case ของ Algorithm A

เกิดขึ้นเมื่ออัลกอริทึมต้องตรวจสอบอาคารทางด้านขวาหลายตำแหน่งสำหรับอาคารแต่ละหลัง เช่น กรณีที่ไม่มีอาคารที่สูงกว่าทางด้านขวา

ตัวอย่าง

Input = [5, 4, 3, 2, 1]

อาคารแต่ละหลังต้องตรวจสอบข้อมูลที่อยู่ทางด้านขวาจนสุด เนื่องจากไม่พบอาคารที่สูงกว่า จำนวนการเปรียบเทียบมีลักษณะเป็น

$(n - 1) + (n - 2) + \dots + 1$

ดังนั้น Worst Case มี Time Complexity เป็น

$O(n^2)$

3.2 Algorithm B: Monotonic Stack

แม้ Algorithm B จะมี WHILE อยู่ภายใน FOR แต่ Worst Case ไม่ได้เป็น $O(n^2)$

เนื่องจากแต่ละ Index ถูก Push เข้า Stack อย่างมากหนึ่งครั้ง และถูก Pop ออกจาก Stack อย่างมากหนึ่งครั้ง

ตัวอย่าง

Input = [1, 2, 3, 4, 5]

เมื่อพบอาคารใหม่ที่สูงกว่า อาจเกิดการ Pop หลาย Index ในรอบเดียว แต่ Index ที่ถูก Pop จะไม่ถูก Pop ซ้ำอีก

ดังนั้นเมื่อพิจารณาจำนวน Operation รวมตลอดการทำงานด้วยแนวคิด **Amortized**

Analysis จะได้

Worst Case Time Complexity = $O(n)$

4. Time Complexity

Time Complexity ใช้สำหรับวิเคราะห์จำนวนขั้นตอนการทำงานของอัลกอริทึมเมื่อจำนวนข้อมูล n เพิ่มขึ้น

Algorithm A: Brute Force

Algorithm A ใช้ Nested Loop โดย Loop ชั้นนอกพิจารณาอาคารแต่ละหลัง และ Loop ชั้นในค้นหาอาคารทางด้านขวา

จึงมีความซับซ้อนดังนี้

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Algorithm B: Monotonic Stack

Algorithm B ใช้ Stack และแต่ละ Index ถูก Push และ Pop อย่างมากหนึ่งครั้ง

จึงมีความซับซ้อนดังนี้

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n)$
Worst Case	$O(n)$

Amortized Analysis ของ Algorithm B

แม้โครงสร้าง Pseudocode จะมี

FOR

WHILE

แต่ไม่สามารถสรุปทันทีว่าเป็น $O(n^2)$ เพราะจำนวนครั้งที่ WHILE ทำงานตลอดทั้งโปรแกรมมีข้อจำกัดสำหรับข้อมูล n อาคาร

- Push รวมไม่เกิน n ครั้ง
- Pop รวมไม่เกิน n ครั้ง

ดังนั้นจำนวน Operation ที่เกี่ยวข้องกับ Stack รวมมีลำดับเป็น $O(n)$

จึงสรุปได้ว่า

Algorithm B มี Time Complexity = $O(n)$

ใน Best Case, Average Case และ Worst Case

5. Space Complexity

5.1 Algorithm A: Brute Force

Algorithm A ไม่ใช้ Stack หรือโครงสร้างข้อมูลเพิ่มเติมในการค้นหา

หากพิจารณา **Auxiliary Space** โดยไม่นับ Array result ซึ่งเป็นพื้นที่สำหรับเก็บ Output จะใช้พื้นที่เพิ่มเติมคงที่

Auxiliary Space = $O(1)$

หากนับพื้นที่ของ result รวมด้วย จะเป็น $O(n)$ เนื่องจาก result มีขนาดเท่ากับจำนวนอาคาร

5.2 Algorithm B: Monotonic Stack

Algorithm B ใช้ Stack สำหรับเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element

ในกรณีเลขร้ายที่สุด Stack สามารถเก็บ Index ของอาคารได้ครบทั้ง n ตำแหน่ง

ตัวอย่างเช่น

Input = [5, 4, 3, 2, 1]

เนื่องจากไม่มีอาคารใหม่ที่สูงกว่ายอด Stack จึงไม่มีการ Pop และ Stack สามารถสะสม Index เพิ่มขึ้นเรื่อย ๆ

ดังนั้น

Auxiliary Space = $O(n)$

6. ตารางเปรียบเทียบ

คุณสมบัติ	Algorithm A: Brute Force	Algorithm B: Monotonic Stack
วิธีการ	Nested Loop	Monotonic Stack
Best Case	$O(n)$	$O(n)$
Average Case	$O(n^2)$	$O(n)$
Worst Case	$O(n^2)$	$O(n)$
Auxiliary Space	$O(1)$	$O(n)$
การเข้าถึงข้อมูลซ้ำ	มากกว่า	น้อยกว่า
ความเข้าใจง่าย	ง่าย	ซับซ้อนกว่า
ประสิทธิภาพเมื่อข้อมูลมีขนาดใหญ่	ต่ำกว่า	สูงกว่า
โครงสร้างข้อมูลเพิ่มเติม	ไม่ใช่ Stack	ใช้ Stack
การขยายไปปัญหาอื่น	ทำได้แต่ไม่มี Stack	สามารถประยุกต์กับ Largest Rectangle in Histogram ได้

จากตารางพบว่า Algorithm A มีข้อดีด้านความเข้าใจง่ายและไม่ต้องใช้ Stack เพิ่มเติม แต่มี Time Complexity ใน Average Case และ Worst Case เป็น $O(n^2)$ จึงเหมาะสำหรับใช้เป็น **Baseline** ในการเปรียบเทียบ ส่วน Algorithm B มีความซับซ้อนในการทำความเข้าใจมากกว่าและต้องใช้พื้นที่สำหรับ Stack แต่สามารถลด Time Complexity ลงเหลือ $O(n)$ ทำให้เหมาะกับข้อมูลที่มีจำนวนมากกว่า

7 Amortized Analysis ของ Algorithm B

แม้ Algorithm B จะมี while loop อยู่ภายใน for loop แต่แต่ละ Index จะถูก Push ลง Stack อย่างมากหนึ่งครั้ง และ Pop อย่างมากหนึ่งครั้ง ดังนั้นจำนวนการดำเนินการของ Stack รวมไม่เกินสัดส่วนของ n ทำให้ Time Complexity โดยรวมเป็น $O(n)$

บทที่ 6

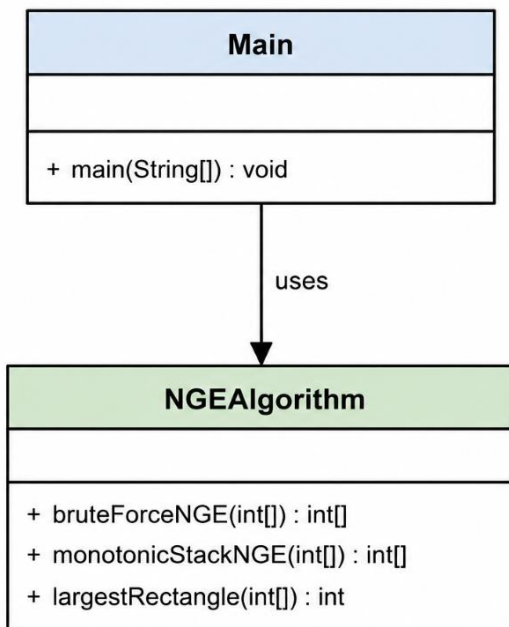
การพัฒนาโปรแกรม Java

บทนี้แนะนำการพัฒนาโปรแกรมสำหรับแก้ปัญหา City Skyline โดยใช้ภาษา Java โปรแกรมประกอบด้วย Algorithm A: Brute Force และ Algorithm B: Monotonic Stack สำหรับการหา Next Greater Element รวมถึงการหาพื้นที่สี่เหลี่ยมต่อเนื่องที่ใหญ่ที่สุดด้วย Largest Rectangle in Histogram โดยโปรแกรมมีการตรวจสอบ Input, จัดการข้อผิดพลาด, นับจำนวน Operation และวัดเวลาในการทำงานของแต่ละอัลกอริทึม

1. Class Diagram

โปรแกรมออกแบบโดยใช้ 2 คลาสหลัก ได้แก่ Main และ NGEAlgorithm เพื่อให้โครงสร้างของโปรแกรมสอดคล้องกับการทำงานจริงและไม่สร้างคลาสที่เกินความจำเป็น ความสัมพันธ์ระหว่างคลาสคือ Main ทำหน้าที่รับข้อมูลจากผู้ใช้และเรียกใช้เมธอดภายในคลาส NGEAlgorithm เพื่อประมวลผลข้อมูล

Class Diagram



ความสัมพันธ์ระหว่างคลาสคือ Main ทำหน้าที่เรียกใช้เมธอดภายในคลาส NGEAlgorithm เพื่อประมวลผลข้อมูลความสูงของอาคาร

หน้าที่ของแต่ละคลาส

Main

ทำหน้าที่เป็นคลาสหลักของโปรแกรม รับจำนวนอาคารและความสูงของอาคารจากผู้ใช้ ตรวจสอบข้อมูลเบื้องต้น เรียกใช้อัลกอริทึม และแสดงผลลัพธ์

NGEAlgorithm

ทำหน้าที่ประมวลผลอัลกอริทึม ได้แก่

bruteForceNGE() สำหรับ Algorithm A

monotonicStackNGE() สำหรับ Algorithm B

largestRectangle() สำหรับปัญหา Largest Rectangle in Histogram

2. โครงสร้างคลาส

2.1 คลาส Main

คลาส Main เป็นคลาสหลักที่ใช้เริ่มต้นการทำงานของโปรแกรม โดยมีเมธอด main() ทำหน้าที่รับข้อมูลจากผู้ใช้ ตรวจสอบข้อมูล เรียกใช้อัลกอริทึม และแสดงผลลัพธ์ ขั้นตอนการทำงานของคลาส Main มีดังนี้

1. สร้าง Scanner สำหรับรับข้อมูลจากผู้ใช้
2. รับจำนวนอาคาร
3. ตรวจสอบว่าจำนวนอาคารเป็นจำนวนเต็มและมากกว่า 0
4. รับค่าความสูงของอาคารแต่ละหลัง
5. ตรวจสอบว่าความสูงเป็นจำนวนเต็มและไม่ติดลบ
6. เรียกใช้ Algorithm A: bruteForceNGE()
7. เรียกใช้ Algorithm B: monotonicStackNGE()
8. เปรียบเทียบผลลัพธ์ของ Algorithm A และ Algorithm B
9. เรียกใช้ largestRectangle()
10. นับจำนวน Operation
11. วัดเวลาในการทำงาน
12. แสดงผลลัพธ์และข้อมูลการเปรียบเทียบ

2.2 คลาส NGEAlgorithm

คลาส NGEAlgorithm เป็นคลาสที่เก็บการทำงานของอัลกอริทึมที่ใช้ในการแก้ปัญหา โดยประกอบด้วย 3 เมธอดหลัก ได้แก่

1. bruteForceNGE(int[] heights)

ใช้วิธี Brute Force แบบ Nested Loop สำหรับค้นหา Next Greater Element ของอาคารแต่ละหลัง โดยไม่ใช้ Stack

2. monotonicStackNGE(int[] heights)

ใช้ Monotonic Stack เก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element เมื่อพบอาคารใหม่ที่สูงกว่ายอด Stack จะทำการ Pop Index ที่เกี่ยวข้องและบันทึกคำตอบ แต่ละ Index จะถูก Push อย่างมากหนึ่งครั้งและ Pop อย่างมากหนึ่งครั้ง จึงมี Time Complexity รวมเป็น $O(n)$

3. largestRectangle(int[] heights)

ใช้ Monotonic Stack เพื่อหาพื้นที่สี่เหลี่ยมต่อเนื่องที่ใหญ่ที่สุดจากความสูงของอาคารที่เรียงกัน โดยมี Time Complexity เป็น $O(n)$ และ Auxiliary Space เป็น $O(n)$

3. เมธอดสำคัญ

คลาส	เมธอด	หน้าที่
Main	main()	รับข้อมูล ตรวจสอบ Input เรียกอัลกอริทึม และแสดงผล
NGEAlgorithm	bruteForceNGE()	หา Next Greater Element ด้วย Brute Force
NGEAlgorithm	monotonicStackNGE()	หา Next Greater Element ด้วย Monotonic Stack
NGEAlgorithm	largestRectangle()	หาพื้นที่สี่เหลี่ยมต่อเนื่องที่ใหญ่ที่สุด

3.1 เมธอด bruteForceNGE()

เมธอดนี้ใช้ Nested Loop โดยพิจารณาอาคารแต่ละหลังจากซ้ายไปขวา และค้นหาอาคารที่อยู่ทางด้านขวาที่ละตำแหน่ง เมื่อพบอาคารที่มีความสูงมากกว่าอาคารปัจจุบัน จะบันทึกค่าความสูงลงใน result และหยุดการค้นหาของอาคารตำแหน่งนั้นทันที หากตรวจสอบอาคารทางด้านขวครบทั้งหมดแล้วไม่พบอาคารที่สูงกว่า ค่าใน result จะยังคงเป็น -1

Time Complexity:

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

3.2 เมธอด monotonicStackNGE()

เมธอดนี้ใช้ Stack สำหรับเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element

เมื่อพบอาคารใหม่ที่มีความสูงมากกว่ายอด Stack โปรแกรมจะทำการ pop() Index ออกจาก Stack และกำหนดความสูงของอาคารปัจจุบันให้เป็น Next Greater Element ของ Index ที่ถูก Pop จากนั้นจึง push() Index ของอาคารปัจจุบันเข้า Stack แต่ละ Index จะถูก Push อย่างมากหนึ่งครั้งและ Pop อย่างมากหนึ่งครั้ง จึงสามารถวิเคราะห์ด้วยแนวคิด Amortized Analysis ได้ว่า

Time Complexity = $O(n)$

ในทุกกรณีของข้อมูล

3.3 เมธอด largestRectangle()

เมธอดนี้ใช้ Monotonic Stack เพื่อหาพื้นที่สี่เหลี่ยมต่อเนื่องที่ใหญ่ที่สุดจากความสูงของอาคาร

เมื่อพบอาคารที่มีความสูงน้อยกว่ายอด Stack โปรแกรมจะ Pop Index ออกจาก Stack และคำนวณพื้นที่ของสี่เหลี่ยมที่สามารถสร้างได้

คำนวณพื้นที่จาก

$\text{Area} = \text{Height} \times \text{Width}$

จากนั้นเปรียบเทียบกับ maxArea และเก็บค่าที่มากที่สุดไว้

Time Complexity:

$O(n)$

Auxiliary Space:

$O(n)$

4. Input Validation

โปรแกรมตรวจสอบข้อมูลที่ผู้ใช้ป้อนก่อนนำไปประมวลผล

เพื่อป้องกันข้อมูลที่ไม่ถูกต้องและลดโอกาสเกิด Runtime Error

การตรวจสอบประกอบด้วย

1. จำนวนอาคารต้องเป็นจำนวนเต็ม
2. จำนวนอาคารต้องมากกว่า 0
3. ความสูงของอาคารต้องเป็นจำนวนเต็ม
4. ความสูงของอาคารต้องไม่ติดลบ
5. ต้องป้อนข้อมูลความสูงครบตามจำนวนอาคาร
6. โปรแกรมต้องตรวจสอบข้อมูลก่อนนำไปใช้กับ Array
7. หากผู้ใช้ป้อนข้อมูลที่ไม่ใช่จำนวนเต็ม

โปรแกรมต้องแจ้งเตือนและไม่ใช้ข้อมูลดังกล่าวในการประมวลผล

ตัวอย่าง Input ที่ไม่ถูกต้อง

จำนวนอาคาร = 0

โปรแกรมจะแจ้งว่า
จำนวนอาคารต้องมากกว่า 0
หรือหากป้อนความสูงเป็น
-5
โปรแกรมจะแจ้งว่า
ความสูงต้องไม่ติดลบ

5 Error Handling

โปรแกรมมีการจัดการข้อผิดพลาดเพื่อให้สามารถรองรับกรณีที่ผู้ใช้ป้อนข้อมูลไม่ถูกต้อง โดยไม่ทำให้โปรแกรมหยุดทำงานอย่างไม่เหมาะสม
การจัดการข้อผิดพลาดประกอบด้วย

1. ตรวจสอบข้อมูล Integer

ใช้ `hasNextInt()` ก่อนรับค่าจำนวนเต็ม เพื่อป้องกันกรณีที่ผู้ใช้ป้อนข้อมูล เช่น abc แทนจำนวนเต็ม

2. ตรวจสอบจำนวนอาคาร

หากจำนวนอาคารน้อยกว่าหรือเท่ากับ 0 โปรแกรมจะไม่ดำเนินการประมวลผล และแจ้งให้ผู้ใช้ป้อนค่าใหม่

3. ตรวจสอบความสูง

หากความสูงของอาคารมีค่าติดลบ โปรแกรมจะแจ้งเตือนและให้ผู้ใช้ป้อนข้อมูลใหม่

4. ตรวจสอบ Stack

ก่อนเรียกใช้ `peek()` หรือ `pop()` โปรแกรมต้องตรวจสอบว่า Stack ไม่ว่าง เพื่อป้องกันการเข้าถึงข้อมูลจาก Stack ที่ไม่มีสมาชิก

5. ตรวจสอบ Array Index

โปรแกรมใช้ Index ที่อยู่ในขอบเขตของ Array เพื่อป้องกัน `ArrayIndexOutOfBoundsException`

การจัดการข้อผิดพลาดเหล่านี้ช่วยให้โปรแกรมสามารถรองรับกรณี Input ไม่ถูกต้องและกรณีขอบเขตต่าง ๆ ที่ต้องใช้ในการสาธิต

6. Source Code ภาษา Java

โปรแกรมพัฒนาด้วยภาษา Java โดยแบ่งออกเป็น 3 ไฟล์ ได้แก่

- Main.java รับจำนวนอาคารและความสูง ตรวจสอบ Input เรียก Algorithm A/B และแสดงผล
- NGEAlgorithm.java ประกอบด้วย `bruteForceNGE()`, `monotonicStackNGE()` และ `largestRectangle()`
- Experiment.java ใช้สำหรับการทดลองประสิทธิภาพของ Algorithm A และ B โดยวัดเวลาและจำนวน Operation

6.1 Source Code: NGEAlgorithm.java

```
import java.util.Stack;
```

```
public class NGEAlgorithm {

    private long operationA;
    private long operationB;

    // =====
    // Algorithm A: Brute Force
    // =====
    public int[] bruteForceNGE(int[] heights) {

        operationA = 0;

        int n = heights.length;
        int[] result = new int[n];

        for (int i = 0; i < n; i++) {

            result[i] = -1;

            for (int j = i + 1; j < n; j++) {

                // เปรียบเทียบความสูง = 1 Operation
                operationA++;

                if (heights[j] > heights[i]) {
                    result[i] = heights[j];
                    break;
                }
            }
        }

        return result;
    }

    // =====
    // Algorithm B: Monotonic Stack
    // =====
    public int[] monotonicStackNGE(int[] heights) {
```

```
operationB = 0;

int n = heights.length;
int[] result = new int[n];

Stack<Integer> stack = new Stack<>();

for (int i = 0; i < n; i++) {

    result[i] = -1;

    while (!stack.isEmpty()) {

        // เปรียบเทียบความสูง = 1 Operation
        operationB++;

        if (heights[i] > heights[stack.peek()]) {

            // Pop = 1 Operation
            int index = stack.pop();
            operationB++;

            result[index] = heights[i];

        } else {
            break;
        }
    }

    // Push = 1 Operation
    stack.push(i);
    operationB++;
}

return result;
}
```

```
// =====  
// Largest Rectangle in Histogram  
// =====  
public int largestRectangle(int[] heights) {  
  
    Stack<Integer> stack = new Stack<>();  
    int maxArea = 0;  
  
    for (int i = 0; i <= heights.length; i++) {  
  
        int currentHeight;  
  
        if (i == heights.length) {  
            currentHeight = 0;  
        } else {  
            currentHeight = heights[i];  
        }  
  
        while (!stack.isEmpty()  
            && currentHeight < heights[stack.peek()]) {  
  
            int height = heights[stack.pop()];  
  
            int width;  
  
            if (stack.isEmpty()) {  
                width = i;  
            } else {  
                width = i - stack.peek() - 1;  
            }  
  
            int area = height * width;  
  
            if (area > maxArea) {  
                maxArea = area;  
            }  
        }  
    }  
}
```

```

        stack.push(i);
    }

    return maxArea;
}

// =====
// Get Operation Count
// =====
public long getOperationA() {
    return operationA;
}

public long getOperationB() {
    return operationB;
}
}

```

6.2 Source Code: Main.java

```

import java.util.Arrays;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        NGEAlgorithm algorithm = new NGEAlgorithm();

        int n;

        // =====
        // รับจำนวนอาคาร
        // =====
        while (true) {

```

```

System.out.print("Enter number of buildings: ");

if (!scanner.hasNextInt()) {
    System.out.println(
        "Invalid input. Please enter an integer.");
    scanner.next();
    continue;
}

n = scanner.nextInt();

if (n <= 0) {
    System.out.println(
        "Number of buildings must be greater than 0.");
    continue;
}

break;
}

// =====
// รับความสูงของอาคาร
// =====
int[] heights = new int[n];

for (int i = 0; i < n; i++) {

    while (true) {

        System.out.print(
            "Enter height of building "
            + (i + 1) + ": ");

        if (!scanner.hasNextInt()) {
            System.out.println(
                "Invalid input. Please enter an integer.");
            scanner.next();
            continue;

```

```

    }

    int height = scanner.nextInt();

    if (height < 0) {
        System.out.println(
            "Height must not be negative.");
        continue;
    }

    heights[i] = height;
    break;
}
}

// =====
// แสดง Input
// =====
System.out.println();
System.out.println(
    "Input: " + Arrays.toString(heights));

// =====
// Algorithm A: Brute Force
// =====
long startA = System.nanoTime();

int[] resultA =
    algorithm.bruteForceNGE(heights);

long endA = System.nanoTime();

long timeA = endA - startA;
long operationsA =
    algorithm.getOperationA();

// =====
// Algorithm B: Monotonic Stack

```

```

// =====
long startB = System.nanoTime();

int[] resultB =
    algorithm.monotonicStackNGE(heights);

long endB = System.nanoTime();

long timeB = endB - startB;
long operationsB =
    algorithm.getOperationB();

// =====
// Largest Rectangle
// =====
int largestRectangle =
    algorithm.largestRectangle(heights);

// =====
// แสดงผล Algorithm A
// =====
System.out.println();
System.out.println(
    "Algorithm A: Brute Force");

System.out.println(
    "Result: " + Arrays.toString(resultA));

System.out.println(
    "Time: " + timeA + " ns");

System.out.println(
    "Operations: " + operationsA);

// =====
// แสดงผล Algorithm B
// =====
System.out.println();

```

```

System.out.println(
    "Algorithm B: Monotonic Stack");

System.out.println(
    "Result: " + Arrays.toString(resultB));

System.out.println(
    "Time: " + timeB + " ns");

System.out.println(
    "Operations: " + operationsB);

// =====
// Largest Rectangle
// =====
System.out.println();
System.out.println(
    "Largest Rectangle: "
    + largestRectangle);

// =====
// เปรียบเทียบผลลัพธ์
// =====
System.out.println();

if (Arrays.equals(resultA, resultB)) {

    System.out.println(
        "Result comparison: PASS");

} else {

    System.out.println(
        "Result comparison: FAIL");

}

scanner.close();
}

```

```
}
```

6.3 Source Code: Experiment.java

```
import java.util.Arrays;
import java.util.Random;

public class Experiment {

    public static void main(String[] args) {

        NGEAlgorithm algorithm = new NGEAlgorithm();

        int[] sizes = {
            10,
            100,
            1000,
            5000,
            10000
        };

        int numberOfRounds = 5;

        System.out.println("=====");
        System.out.println("      NGE ALGORITHM EXPERIMENT");
        System.out.println("      5 ROUNDS PER DATA SIZE");
        System.out.println("=====");

        System.out.println();

        for (int n : sizes) {

            // สร้างข้อมูลแบบสุ่ม
            int[] heights = generateRandomData(n);

            // =====
            // Warm-up
            // =====
        }
    }
}
```

```
for (int i = 0; i < 3; i++) {  
    algorithm.bruteForceNGE(heights);  
    algorithm.monotonicStackNGE(heights);  
}  
  
long totalTimeA = 0;  
long totalTimeB = 0;  
  
long operationsA = 0;  
long operationsB = 0;  
  
int[] resultA = null;  
int[] resultB = null;  
  
// =====  
// ทดลอง 5 รอบ  
// =====  
for (int round = 1; round <= numberOfRounds; round++) {  
  
    // Algorithm A  
    long startA = System.nanoTime();  
  
    resultA = algorithm.bruteForceNGE(heights);  
  
    long endA = System.nanoTime();  
  
    totalTimeA += (endA - startA);  
  
    operationsA = algorithm.getOperationA();  
  
    // Algorithm B  
    long startB = System.nanoTime();  
  
    resultB = algorithm.monotonicStackNGE(heights);  
  
    long endB = System.nanoTime();  

```

```

        totalTimeB += (endB - startB);

        operationsB = algorithm.getOperationB();
    }

    // =====
    // ค่าเฉลี่ยเวลา
    // =====
    long averageTimeA =
        totalTimeA / numberOfRounds;

    long averageTimeB =
        totalTimeB / numberOfRounds;

    // =====
    // ตรวจสอบผลลัพธ์
    // =====
    boolean sameResult =
        Arrays.equals(resultA, resultB);

    // =====
    // แสดงผล
    // =====
    System.out.println("n = " + n);

    System.out.println(
        "Algorithm A Average Time: "
        + averageTimeA + " ns"
    );

    System.out.println(
        "Algorithm A Operations: "
        + operationsA
    );

    System.out.println(
        "Algorithm B Average Time: "
        + averageTimeB + " ns"
    );

```

```

    );

    System.out.println(
        "Algorithm B Operations: "
        + operationsB
    );

    System.out.println(
        "Result Comparison: "
        + (sameResult ? "PASS" : "FAIL")
    );

    System.out.println();
}

System.out.println("=====");
System.out.println("Experiment completed.");
System.out.println("=====");
}

// =====
// สร้างข้อมูลสุ่ม
// =====
private static int[] generateRandomData(int n) {

    Random random = new Random(42);

    int[] heights = new int[n];

    for (int i = 0; i < n; i++) {

        heights[i] =
            random.nextInt(1000) + 1;
    }

    return heights;
}
}

```

Experiment.java เป็นคลาสที่ใช้สำหรับการทดลองประสิทธิภาพของ Algorithm A: Brute Force และ Algorithm B: Monotonic Stack โดยทดสอบข้อมูลหลายขนาดและทำการทดลอง 5 รอบต่อขนาดข้อมูล จากนั้นคำนวณเวลาเฉลี่ยและจำนวน Operation เพื่อนำผลลัพธ์ไปเปรียบเทียบประสิทธิภาพและจัดทำกราฟ

6.4 การนับจำนวน Operation

เพื่อใช้เปรียบเทียบประสิทธิภาพของ Algorithm A: Brute Force และ Algorithm B: Monotonic Stack โปรแกรมมีการนับจำนวน Operation ที่เกิดขึ้นระหว่างการประมวลผล โดยกำหนดรูปแบบการนับให้สอดคล้องกับลักษณะการทำงานของแต่ละอัลกอริทึม ดังนี้

Algorithm A: Brute Force

Algorithm A ใช้วิธี **Nested Loop** ในการค้นหา Next Greater Element

โดยตรวจสอบอาคารทางด้านขวาที่ละตำแหน่ง กำหนดให้

การเปรียบเทียบความสูงของอาคารแต่ละครั้ง = **1 Operation**

ดังนั้นจำนวน Operation ของ Algorithm A จะขึ้นอยู่กับจำนวนครั้งที่ต้องเปรียบเทียบอาคารในแต่ละตำแหน่ง หากต้องตรวจสอบอาคารทางด้านขวาหลายตำแหน่ง จำนวน Operation จะเพิ่มขึ้นตามจำนวนการเปรียบเทียบ

Algorithm B: Monotonic Stack

Algorithm B ใช้ **Stack** สำหรับเก็บ Index ของอาคารที่ยังไม่พบ Next Greater Element และรักษาคุณสมบัติของ Monotonic Stack

กำหนดให้

- การเปรียบเทียบความสูงของอาคาร = **1 Operation**
- การ push Index ลงใน Stack = **1 Operation**
- การ pop Index ออกจาก Stack = **1 Operation**

ดังนั้นจำนวน Operation ของ Algorithm B จะประกอบด้วยจำนวนการเปรียบเทียบ รวมกับจำนวนครั้งที่มีการ push และ pop ข้อมูลใน Stack

บทที่ 7

การทดลอง

1. วัตถุประสงค์ของการทดลอง

การทดลองนี้มีวัตถุประสงค์เพื่อเปรียบเทียบประสิทธิภาพของ **Algorithm A: Brute Force** และ **Algorithm B: Monotonic Stack** สำหรับการหา **Next Greater Element (NGE)** โดยพิจารณาจากความถูกต้องของผลลัพธ์ เวลาในการประมวลผล และจำนวน Operation ที่เกิดขึ้นระหว่างการทำงาน นอกจากนี้ยังทดสอบโปรแกรมตามกรณีทดสอบที่กำหนด ได้แก่ กรณีตึกสูงขึ้นต่อเนื่อง ตึกเตี้ยลงต่อเนื่อง ตึกสูงเท่ากันทั้งหมด ตึกสลับสูง-ต่ำ มีตึกเดียว ไม่มีตึกที่สูงกว่าทางด้านขวา ตึกสูงที่สุดอยู่ตำแหน่งแรก และตึกสูงที่สุดอยู่ตำแหน่งสุดท้าย รวมถึงกรณีปกติและ Input ที่ไม่ถูกต้อง เพื่อยืนยันว่าโปรแกรมสามารถทำงานได้ตามข้อกำหนดของโจทย์

2. ขนาดข้อมูล

ในการทดลองประสิทธิภาพ กำหนดขนาดข้อมูลของอาคารหลายระดับ เพื่อศึกษาพฤติกรรมของ **Algorithm A: Brute Force** และ **Algorithm B: Monotonic Stack** เมื่อจำนวนข้อมูลเพิ่มขึ้น

กำหนดขนาดข้อมูลดังนี้

ชุดทดลอง	จำนวนอาคาร (n)
ชุดที่ 1	10
ชุดที่ 2	100
ชุดที่ 3	1,000
ชุดที่ 4	5,000
ชุดที่ 5	10,000

ข้อมูลที่ใช้ในการทดลองเป็นข้อมูลแบบสุ่ม โดยใช้ข้อมูลชุดเดียวกันในการทดสอบ Algorithm A และ Algorithm B ในแต่ละขนาดข้อมูล เพื่อให้สามารถเปรียบเทียบประสิทธิภาพของทั้งสอง Algorithm ได้อย่างเหมาะสม

3. จำนวนรอบทดลอง

เพื่อให้ผลการทดลองมีความน่าเชื่อถือมากขึ้น โปรแกรมกำหนดให้แต่ละขนาดข้อมูลทำการทดลอง **5 รอบ** ในการทดลองบันทึกข้อมูลดังต่อไปนี้

- ขนาดข้อมูล N
- เวลาเฉลี่ยของ Algorithm A
- จำนวน Operation ของ Algorithm A
- เวลาเฉลี่ยของ Algorithm B
- จำนวน Operation ของ Algorithm B
- ผลลัพธ์ของ Algorithm A และ Algorithm B
- ผลการเปรียบเทียบความถูกต้องของผลลัพธ์

ผลการทดลองมีดังนี้

1. $n = 10$

```
n = 10
Algorithm A Average Time: 560 ns
Algorithm A Operations: 16
Algorithm B Average Time: 6360 ns
Algorithm B Operations: 28
Result Comparison: PASS
```

ผลการทดลองพบว่า Algorithm A และ Algorithm B ให้ผลลัพธ์ตรงกัน โดยมีผลการเปรียบเทียบเป็น PASS

2. $n = 100$

```
n = 100
Algorithm A Average Time: 14160 ns
Algorithm A Operations: 416
Algorithm B Average Time: 101820 ns
Algorithm B Operations: 386
Result Comparison: PASS
```

ผลการทดลองพบว่า Algorithm A และ Algorithm B ให้ผลลัพธ์ตรงกัน โดยมีผลการเปรียบเทียบเป็น PASS

3. $n = 1,000$

```
n = 1000
Algorithm A Average Time: 130560 ns
Algorithm A Operations: 7033
Algorithm B Average Time: 318960 ns
Algorithm B Operations: 3978
Result Comparison: PASS
```

ผลการทดลองแสดงว่า Algorithm A และ Algorithm B สามารถประมวลผลข้อมูลขนาด 1,000 รายการได้
โดยผลการเปรียบเทียบเป็น PASS

4. $n = 5,000$

```
n = 5000
Algorithm A Average Time: 317260 ns
Algorithm A Operations: 49890
Algorithm B Average Time: 1092160 ns
Algorithm B Operations: 19958
Result Comparison: PASS
```

ผลการทดลองแสดงว่า Algorithm A และ Algorithm B สามารถประมวลผลข้อมูลขนาด 5,000 รายการได้
โดยผลการเปรียบเทียบเป็น PASS

5. $n = 10,000$

```
n = 10000
Algorithm A Average Time: 251460 ns
Algorithm A Operations: 134980
Algorithm B Average Time: 1898860 ns
Algorithm B Operations: 39954
Result Comparison: PASS
```

ผลการทดลองแสดงว่า Algorithm A และ Algorithm B สามารถประมวลผลข้อมูลขนาด 10,000 รายการได้ โดยผลการเปรียบเทียบเป็น PASS

ในการทดลองนี้จึงนำค่าเวลาเฉลี่ยและจำนวน Operation

ที่ได้จากแต่ละขนาดข้อมูลมาใช้ในการประกอบการวิเคราะห์และเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B เพื่อให้การเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B มีความน่าเชื่อถือ จึงกำหนดให้ทำการทดลองซ้ำ 5 รอบในแต่ละขนาดข้อมูล โดยเวลาในการทำงานวัดด้วย System.nanoTime() และนำผลจากการทดลองทั้ง 5 รอบมาคำนวณเป็นค่าเฉลี่ย

4 กรณีทดสอบ

การทดสอบในส่วนนี้เป็นการตรวจสอบความถูกต้องของโปรแกรมตามกรณีทดสอบที่กำหนดในโจทย์ โดยใช้ข้อมูลเดียวกันในการทดสอบ Algorithm A และ Algorithm B และตรวจสอบว่าผลลัพธ์ของทั้งสอง Algorithm ตรงกันหรือไม่

4.1 กรณีปกติ (Normal Case)

กำหนด Input เป็น

Input: [4, 5, 2, 10, 8]

ผลลัพธ์:

```
Input: [4, 5, 2, 10, 8]

Algorithm A: Brute Force
Result: [5, 10, 10, -1, -1]
Time: 50500 ns
Operations: 5

Algorithm B: Monotonic Stack
Result: [5, 10, 10, -1, -1]
Time: 477500 ns
Operations: 13

Largest Rectangle: 16

Result comparison: PASS
```

Algorithm A: [5, 10, 10, -1, -1]

Algorithm B: [5, 10, 10, -1, -1]

ผลลัพธ์ของทั้งสอง Algorithm ตรงกัน จึงถือว่าการทดสอบผ่าน

Result comparison: PASS

4.1.1 ตีกลงขึ้นต่อเนื่อง (Increasing)

กำหนด Input เป็น

Input: [1, 2, 3, 4, 5]

ผลลัพธ์:

```

Input: [1, 2, 3, 4, 5]

Algorithm A: Brute Force
Result: [2, 3, 4, 5, -1]
Time: 6900 ns
Operations: 4

Algorithm B: Monotonic Stack
Result: [2, 3, 4, 5, -1]
Time: 3347200 ns
Operations: 13

Largest Rectangle: 9

Result comparison: PASS

```

Algorithm A: [2, 3, 4, 5, -1]

Algorithm B: [2, 3, 4, 5, -1]

ในกรณีนี้อาคารแต่ละหลังจะพบอาคารที่สูงกว่าทางด้านขวาเป็นอาคารถัดไป
ยกเว้นอาคารสุดท้ายที่ไม่มีอาคารทางด้านขวา

Result comparison: PASS

กรณีนี้เหมาะสำหรับใช้แสดง **Best Case** ของ Algorithm A เนื่องจากอาคารส่วนใหญ่สามารถพบ Next Greater Element ได้จากตำแหน่งถัดไปทันที

4.3 ตีกลงต่อเนื่อง (Decreasing)

กำหนด Input เป็น

Input: [5, 4, 3, 2, 1]

ผลลัพธ์:

```

Input: [5, 4, 3, 2, 1]

Algorithm A: Brute Force
Result: [-1, -1, -1, -1, -1]
Time: 8000 ns
Operations: 10

Algorithm B: Monotonic Stack
Result: [-1, -1, -1, -1, -1]
Time: 477400 ns
Operations: 9

Largest Rectangle: 9

Result comparison: PASS

```

Algorithm A: [-1, -1, -1, -1, -1]

Algorithm B: [-1, -1, -1, -1, -1]

ในกรณีนี้ไม่มีอาคารทางด้านขวาที่สูงกว่าอาคารปัจจุบัน ดังนั้นทุกตำแหน่งจึงมีผลลัพธ์เป็น -1 Result comparison: PASS

กรณีนี้เป็นกรณีที่เหมาะสำหรับแสดง Worst Case ของ Algorithm A

เนื่องจากต้องตรวจสอบอาคารทางด้านขวาหลายตำแหน่งก่อนสรุปว่าไม่พบ Next Greater Element

4.4 ตึกสูงเท่ากันทั้งหมด

กำหนด Input เป็น

Input:

[5, 5, 5, 5]

ผลลัพธ์:

```
Input: [5, 5, 5, 5]

Algorithm A: Brute Force
Result: [-1, -1, -1, -1]
Time: 11200 ns
Operations: 6

Algorithm B: Monotonic Stack
Result: [-1, -1, -1, -1]
Time: 525400 ns
Operations: 7

Largest Rectangle: 20

Result comparison: PASS
```

Algorithm A: [-1, -1, -1, -1]

Algorithm B: [-1, -1, -1, -1]

เนื่องจากค่าที่เท่ากันไม่ถือว่าเป็นค่าที่สูงกว่า จึงไม่มี Next Greater Element สำหรับทุกตำแหน่ง

Result comparison: PASS

4.5 ตึกสลับสูง-ต่ำ

กำหนด Input เป็น

Input: [2, 8, 3, 7, 1, 6]

ผลลัพธ์:

```
Input: [2, 8, 3, 7, 1, 6]

Algorithm A: Brute Force
Result: [8, -1, 7, -1, 6, -1]
Time: 37000 ns
Operations: 9

Algorithm B: Monotonic Stack
Result: [8, -1, 7, -1, 6, -1]
Time: 3088300 ns
Operations: 16

Largest Rectangle: 9

Result comparison: PASS
```

Algorithm A: [8, -1, 7, -1, 6, -1]

Algorithm B: [8, -1, 7, -1, 6, -1]

กรณีนี้ใช้ทดสอบการทำงานของ Algorithm A และ Algorithm B

เมื่อข้อมูลมีรูปแบบความสูงสลับระหว่างอาคารสูงและอาคารต่ำ

Result comparison: PASS

4.6 มีตึกเดียว

กำหนด Input เป็น

Input: [10]

ผลลัพธ์:

```
Input: [10]

Algorithm A: Brute Force
Result: [-1]
Time: 8600 ns
Operations: 0

Algorithm B: Monotonic Stack
Result: [-1]
Time: 587500 ns
Operations: 1

Largest Rectangle: 10

Result comparison: PASS
```

Algorithm A: [-1]

Algorithm B: [-1]

เนื่องจากมีอาคารเพียงหนึ่งหลังและไม่มีอาคารทางด้านขวา จึงไม่สามารถหา Next Greater Element ได้

Result comparison: PASS

4.7 ไม่มีตึกที่สูงกว่าทางด้านขวา

กำหนด Input เป็น

Input: [10, 8, 6, 4, 2]

ผลลัพธ์:

```
Input: [10, 8, 6, 4, 2]

Algorithm A: Brute Force
Result: [-1, -1, -1, -1, -1]
Time: 7000 ns
Operations: 10

Algorithm B: Monotonic Stack
Result: [-1, -1, -1, -1, -1]
Time: 2405700 ns
Operations: 9

Largest Rectangle: 18

Result comparison: PASS
```

Algorithm A: [-1, -1, -1, -1, -1]

Algorithm B: [-1, -1, -1, -1, -1]

ในกรณีนี้ไม่มีอาคารใดทางด้านขวาที่มีความสูงมากกว่าอาคารปัจจุบัน จึงได้ค่า -1 ทุกตำแหน่ง

Result comparison: PASS

4.8 ตึกสูงที่สุดอยู่ตำแหน่งแรก

กำหนด Input เป็น

Input: [10, 8, 6, 4, 2]

ผลลัพธ์:

```
Input: [10, 8, 6, 4, 2]

Algorithm A: Brute Force
Result: [-1, -1, -1, -1, -1]
Time: 30700 ns
Operations: 10

Algorithm B: Monotonic Stack
Result: [-1, -1, -1, -1, -1]
Time: 598500 ns
Operations: 9

Largest Rectangle: 18

Result comparison: PASS
```

Algorithm A: [-1, -1, -1, -1, -1]

Algorithm B: [-1, -1, -1, -1, -1]

อาคารที่มีความสูงมากที่สุดอยู่ตำแหน่งแรก จึงไม่มีอาคารทางด้านขวาที่สูงกว่า นอกจากนี้อาคารตำแหน่งอื่นก็ไม่มีอาคารที่สูงกว่าทางด้านขวาเช่นกัน

Result comparison: PASS

หมายเหตุ: กรณีนี้มี Input เหมือนกับกรณี 7.4.7 ได้ เพราะทั้งสองกรณีให้ผลลัพธ์ NGE เป็น -1 ทั้งหมด แต่ใช้เพื่อนั่น เงื่อนไขการวางตำแหน่งของอาคารที่สูงที่สุด ตามข้อกำหนดของโจทย์

4.9 ตึกสูงที่สุดอยู่ตำแหน่งสุดท้าย

กำหนด Input เป็น

Input: [2, 3, 4, 5, 10]

ผลลัพธ์:

```
Input: [2, 3, 4, 5, 10]

Algorithm A: Brute Force
Result: [3, 4, 5, 10, -1]
Time: 8600 ns
Operations: 4

Algorithm B: Monotonic Stack
Result: [3, 4, 5, 10, -1]
Time: 2132600 ns
Operations: 13

Largest Rectangle: 12

Result comparison: PASS
```

Algorithm A: [3, 4, 5, 10, -1]

Algorithm B: [3, 4, 5, 10, -1]

อาคารส่วนใหญ่สามารถพบ Next Greater Element ได้

โดยอาคารที่สูงที่สุดซึ่งอยู่ตำแหน่งสุดท้ายไม่มีอาคารทางด้านขวาที่สูงกว่า **Result**
comparison: PASS

4.10 ข้อมูลว่างและ Stack ว่าง

สำหรับ Algorithm B จะมีการสร้าง Stack ในสถานะว่างก่อนเริ่มการประมวลผล และมีการตรวจสอบ isEmpty() ก่อนการเข้าถึงข้อมูลด้วย peek() หรือ pop() เพื่อป้องกันการเข้าถึงข้อมูลจาก Stack ที่ไม่มีสมาชิก

อย่างไรก็ตาม Main.java กำหนดให้จำนวนอาคารต้องมากกว่า 0 ดังนั้นกรณี $n = 0$ จะถูกปฏิเสธก่อนเข้าสู่ Algorithm กรณีนี้จึงใช้สำหรับอธิบายการจัดการข้อมูลว่างและการป้องกันการใช้งาน Stack ที่ว่าง

4.11 Input ไม่ถูกต้อง

โปรแกรมมีการตรวจสอบ Input ก่อนนำข้อมูลไปประมวลผล โดยทดสอบกรณีต่าง ๆ ดังนี้

กรณีที่ 1: จำนวนอาคารเป็น 0

เมื่อป้อน Enter number of buildings: 0

โปรแกรมแสดง Number of buildings must be greater than 0.

```
Enter number of buildings: 0
Number of buildings must be greater than 0.
Enter number of buildings: █
```

และให้ผู้ใช้ป้อนข้อมูลใหม่

กรณีที่ 2: ป้อนข้อความแทนจำนวนเต็ม

เมื่อป้อน Enter number of buildings: abc

โปรแกรมแสดง Invalid input. Please enter an integer.

```
Enter number of buildings: abc
Invalid input. Please enter an integer.
Enter number of buildings: █
```

และให้ผู้ใช้ป้อนข้อมูลใหม่

กรณีที่ 3: ความสูงติดลบ

เมื่อป้อนค่าความสูงเป็น -5

โปรแกรมแสดง Height must not be negative.

```
Enter number of buildings: 3
Enter height of building 1: -5
Height must not be negative.
Enter height of building 1: █
```

และให้ผู้ใช้ป้อนค่าใหม่ จากการทดสอบพบว่าโปรแกรมสามารถตรวจสอบและจัดการ Input ที่ไม่ถูกต้องตามเงื่อนไขที่กำหนดไว้

5. ตารางผลการทดลองประสิทธิภาพ

ผลการทดลองประสิทธิภาพจาก Experiment.java ซึ่งทำการทดลอง 5 รอบต่อขนาดข้อมูล และคำนวณเวลาเฉลี่ยของการทำงานของแต่ละ Algorithm ได้ผลดังตารางต่อไปนี้ โดยใช้ข้อมูลชุดเดียวกันในการเปรียบเทียบ Algorithm A และ Algorithm B และตรวจสอบความถูกต้องของผลลัพธ์ด้วย Result comparison ได้ผลดังนี้

ขนาดข้อมูล (n)	Algorithm A เวลาเฉลี่ย (ns)	Algorithm A Operations	Algorithm B เวลาเฉลี่ย (ns)	Algorithm B Operations	ผลลัพธ์
10	500	16	6,140	28	PASS
100	9,200	416	62,760	386	PASS
1,000	124,660	7,033	341,080	3,978	PASS
5,000	329,180	49,890	821,480	19,958	PASS
10,000	181,440	134,980	1,828,500	39,954	PASS

จากตารางพบว่า ผลลัพธ์ของ Algorithm A และ Algorithm B ตรงกันทุกขนาดข้อมูล ตั้งแต่ $n = 10$ จนถึง $n = 10,000$ โดยผลการเปรียบเทียบแสดงเป็น PASS

ในด้านจำนวน Operation พบว่า Algorithm B มีจำนวน Operation ต่ำกว่า Algorithm A เมื่อขนาดข้อมูลเพิ่มขึ้น ซึ่งสอดคล้องกับแนวคิดของ Monotonic Stack ที่แต่ละ Index จะถูก Push และ Pop อย่างมากหนึ่งครั้ง

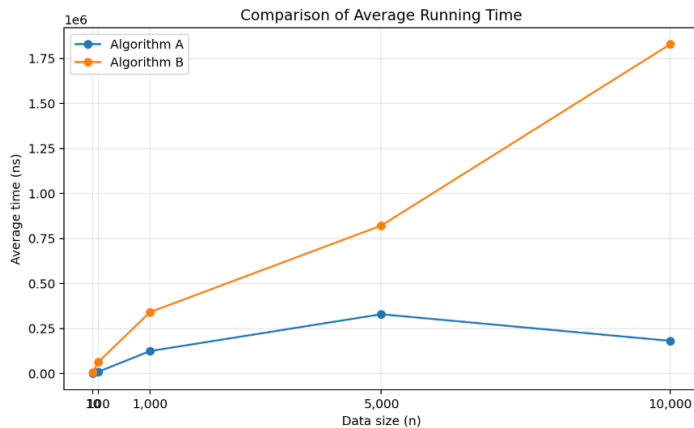
สำหรับเวลาในการประมวลผล พบว่าเวลาไม่ได้เพิ่มขึ้นอย่างเรียบตรงทุกขนาดข้อมูล เช่น Algorithm A ที่ $n = 10,000$

มีเวลาเฉลี่ยต่ำกว่า $n = 5,000$ ความแตกต่างดังกล่าวอาจเกิดจากสภาพแวดล้อมขณะรันโปรแกรม เช่น การทำงานของ JVM

และทรัพยากรของเครื่องคอมพิวเตอร์ ดังนั้นการประเมินประสิทธิภาพจึงควรพิจารณาร่วมกับ Time Complexity และจำนวน Operation ไม่ควรใช้เวลาในการประมวลผลจากการทดลองเพียงค่าเดียวในการสรุป

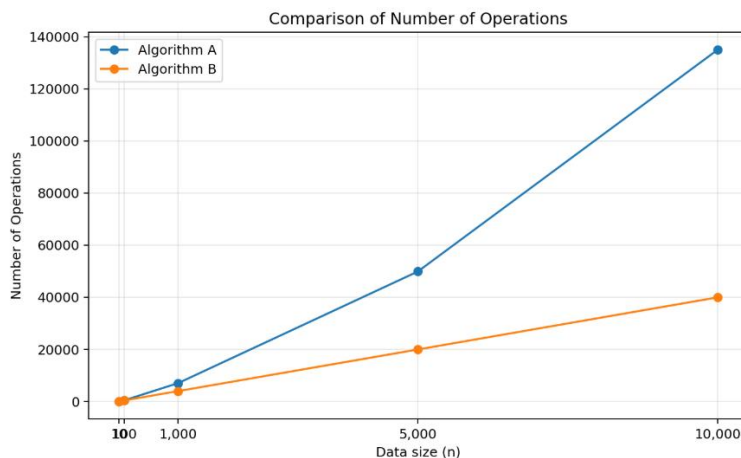
6. กราฟ

กราฟที่ 1 การเปรียบเทียบเวลาในการทำงานของ Algorithm A และ Algorithm B



จากกราฟพบว่า เมื่อขนาดข้อมูลเพิ่มขึ้น เวลาในการทำงานของ Algorithm B มีแนวโน้มสูงกว่า Algorithm A อย่างชัดเจน อย่างไรก็ตาม เวลาในการทำงานจริงอาจมีความผันผวนตามสภาพแวดล้อมของ JVM และเครื่องคอมพิวเตอร์ที่ใช้ทดลอง ดังนั้นจึงควรพิจารณาร่วมกับจำนวน Operation และ Time Complexity โดย Algorithm A มี Time Complexity เป็น $O(n^2)$ ขณะที่ Algorithm B มี Time Complexity เป็น $O(n)$

กราฟที่ 2 การเปรียบเทียบจำนวน Operation ของ Algorithm A และ Algorithm B



จากกราฟพบว่าเมื่อขนาดข้อมูลเพิ่มขึ้น จำนวน Operation ของ Algorithm A เพิ่มขึ้นมากกว่า Algorithm B อย่างชัดเจน โดยเฉพาะเมื่อข้อมูลมีขนาดใหญ่ ซึ่งสอดคล้องกับ Time Complexity ของ Algorithm A ที่เป็น $O(n^2)$

ขณะที่ Algorithm B มี Time Complexity เป็น $O(n)$ แสดงให้เห็นว่า Algorithm B สามารถลดจำนวนการตรวจสอบที่ไม่จำเป็นและเหมาะกับข้อมูลขนาดใหญ่มากกว่า

7 การอภิปรายผล

จากผลการทดลองพบว่า Algorithm A: Brute Force และ Algorithm B: Monotonic Stack ให้ผลลัพธ์ Next Greater Element ตรงกันทุกชุดข้อมูล โดยมีผลการเปรียบเทียบเป็น PASS เมื่อขนาดข้อมูลเพิ่มขึ้น จำนวน Operation ของ Algorithm A มีแนวโน้มเพิ่มขึ้นมากกว่า Algorithm B เนื่องจาก Algorithm A ใช้ Nested Loop จึงมี Time Complexity เป็น $O(n^2)$ ขณะที่ Algorithm B ใช้ Monotonic Stack ซึ่งมี Time Complexity เป็น $O(n)$ ดังนั้น Algorithm B เหมาะสมกับข้อมูลขนาดใหญ่กว่า

เนื่องจากมีประสิทธิภาพเชิงอัลกอริทึมที่ดีกว่าและมีจำนวน Operation น้อยกว่าในชุดการทดลองส่วนใหญ่ อย่างไรก็ตาม เวลาในการรันจริงอาจแตกต่างกันตามสภาพแวดล้อมของ JVM และทรัพยากรของเครื่องคอมพิวเตอร์ที่ใช้ทดลอง

บทที่ 8 สรุป

1. วิธีที่เหมาะสมกว่า

จากการเปรียบเทียบ Algorithm A: Brute Force และ Algorithm B: Monotonic Stack พบว่า Algorithm B: Monotonic Stack เป็นวิธีที่เหมาะสมกว่า โดยเฉพาะเมื่อมีข้อมูลจำนวนมาก เนื่องจากมี Time Complexity เป็น $O(n)$ ขณะที่ Algorithm A มี Time Complexity เป็น $O(n^2)$ จากผลการทดลองเมื่อขนาดข้อมูลเพิ่มขึ้น Algorithm B มีจำนวน Operation น้อยกว่า Algorithm A ในข้อมูลขนาดใหญ่ และยังให้ผลลัพธ์ตรงกับ Algorithm A ทุกชุดการทดลอง

2. เหตุผลในการเลือก

เลือก Algorithm B: Monotonic Stack

เนื่องจากสามารถลดจำนวนการตรวจสอบที่ไม่จำเป็นในการค้นหา Next Greater Element โดยใช้ Stack ในการเก็บ Index ของอาครที่ยังไม่พบอาครที่สูงกว่า

นอกจากนี้ Algorithm B มี Time Complexity เป็น $O(n)$ จึงเหมาะสำหรับการประมวลผลข้อมูลขนาดใหญ่ และมีประสิทธิภาพเชิงอัลกอริทึมดีกว่า Algorithm A ที่มี Time Complexity เป็น $O(n^2)$

นอกจากนี้ จากผลการทดลองในบทที่ 7 พบว่า Algorithm A และ Algorithm B ให้ผลลัพธ์ Next Greater Element ตรงกันทุกชุดข้อมูล โดยมีผลการเปรียบเทียบเป็น PASS และเมื่อขนาดข้อมูลเพิ่มขึ้น Algorithm B มีจำนวน Operation ที่เหมาะสมกว่าในภาพรวม จึงสนับสนุนการเลือกใช้ Monotonic Stack สำหรับข้อมูลขนาดใหญ่

3. ข้อจำกัด

ข้อจำกัดของโปรแกรมและการทดลองมีดังนี้

- โปรแกรมรับข้อมูลผ่านทาง Console ทำให้การป้อนข้อมูลจำนวนมากทำได้ไม่สะดวก
- เวลาในการทำงานที่วัดด้วย System.nanoTime() อาจแตกต่างกันในแต่ละรอบ เนื่องจากขึ้นอยู่กับสภาพแวดล้อมของเครื่องและการทำงานของ Java Virtual Machine
- การทดลองใช้ขนาดข้อมูลที่กำหนดไว้ ได้แก่ 10, 100, 1,000, 5,000 และ 10,000 อาคร จึงยังไม่ได้ทดสอบข้อมูลที่มีขนาดใหญ่กว่านี้
- การนับ Operation เป็นรูปแบบที่กำหนดขึ้นเพื่อใช้เปรียบเทียบ Algorithm A และ Algorithm B ดังนั้นไม่ได้หมายถึงจำนวนคำสั่งของ CPU จริงทั้งหมด

4. แนวทางพัฒนาต่อ

สามารถพัฒนาโปรแกรมเพิ่มเติมได้ดังนี้

- เพิ่มการรองรับการนำเข้าข้อมูลจากไฟล์ เพื่อให้สามารถทดลองกับข้อมูลจำนวนมากได้สะดวกขึ้น
- เพิ่มการแสดงสถานะของ Stack ในแต่ละขั้นตอน เพื่อช่วยให้เห็นการทำงานของ Monotonic Stack ได้ชัดเจนขึ้น
- เพิ่มขนาดและจำนวนชุดข้อมูลในการทดลอง เพื่อให้สามารถวิเคราะห์ประสิทธิภาพได้ละเอียดมากขึ้น
- เพิ่มการสรุปผลการทดลองและสร้างกราฟโดยอัตโนมัติ
- ปรับปรุงส่วนติดต่อผู้ใช้ให้ใช้งานง่ายขึ้น

5. สิ่งที่ได้เรียนรู้

จากการจัดทำโครงงานนี้ ได้เรียนรู้เกี่ยวกับการออกแบบและวิเคราะห์อัลกอริทึม โดยเฉพาะการเปรียบเทียบวิธี Brute Force และ Monotonic Stack สำหรับปัญหา Next Greater Element นอกจากนี้ยังได้เรียนรู้การใช้งาน Stack, การเขียน Pseudocode และ Flowchart, การวิเคราะห์ Time Complexity และ Space Complexity, การเขียนโปรแกรมด้วยภาษา Java, การตรวจสอบ Input และการจัดการข้อผิดพลาด รวมถึงการออกแบบการทดลองและวิเคราะห์ผลจากเวลาและจำนวน Operation

คำถามวิเคราะห์

1. เพราะเหตุใด Algorithm B จึงมี Time Complexity รวม $O(n)$ ทั้งที่มี Loop ซ้อนอยู่ภายใน (ต้องอธิบายด้วยแนวคิด Amortized Analysis)

ตอบ Algorithm B ใช้ for loop เพื่อพิจารณาอาคารแต่ละหลัง และใช้ while loop เพื่อ Pop Index ออกจาก Stack เมื่อพบอาคารใหม่ที่มีความสูงมากกว่ายอด Stack แม้จะมี Loop ซ้อนกัน แต่ while loop ไม่ได้ทำงานครบ n ครั้งสำหรับทุก Index ตามหลัก **Amortized Analysis** แต่ละ Index ของอาคารจะถูกดำเนินการกับ Stack อย่างจำกัด โดย Push อย่างมาก 1 ครั้ง และ Pop อย่างมาก 1 ครั้ง ตลอดการประมวลผล เมื่อ Index ถูก Pop ออกจาก Stack แล้ว จะไม่ถูกนำกลับมาใช้ใน Stack อีก ดังนั้น แม้ในบางรอบ while loop อาจทำงานหลายครั้งติดต่อกัน แต่การ Pop หลายครั้งนั้นเป็นการทำงานที่สะสมมาจาก Index ที่เคยถูก Push ไว้ก่อนหน้านี้ เมื่อพิจารณาการทำงาน **รวมตลอดทั้ง Algorithm** จำนวนครั้งของการ Push และ Pop จึงเป็น $O(n)$ ไม่ใช่ $O(n^2)$ กล่าวอีกแบบคือ for loop มีการประมวลผล n ครั้ง และการทำงานทั้งหมดของ while loop ตลอดทุก iteration รวมกันไม่เกินจำนวน Index ที่ถูก Push และ Pop ซึ่งแต่ละ Index มีจำนวนครั้งจำกัด ดังนั้นเวลาการทำงานรวมจึงเป็น $O(n) + O(n) = O(n)$ จึงสรุปได้ว่า Algorithm B มี Time Complexity รวมเป็น $O(n)$ แม้ว่าจะมี while loop ซ้อนอยู่ภายใน for loop เพราะการวิเคราะห์ต้องพิจารณา จำนวนการทำงานรวมของแต่ละ Index ตลอดการประมวลผล (Amortized Analysis) ไม่ใช่นำจำนวนรอบสูงสุดของ Loop แต่ละตัวมาคูณกันโดยตรง

2. แต่ละ Index ถูก Push และ Pop ได้อย่างมากกี่ครั้ง เพราะเหตุใด

ตอบ แต่ละ Index ถูก Push ได้อย่างมาก 1 ครั้ง และ Pop ได้อย่างมาก 1 ครั้ง เพราะ Index จะถูกเพิ่มเข้า Stack เมื่อยังไม่พบ Next Greater Element และเมื่อถูก Pop ออกแล้ว จะไม่ถูกนำกลับเข้า Stack อีกในการประมวลผลชุดข้อมูลเดิม

3. Algorithm A มี Worst Case Time Complexity เท่าใด และเกิดขึ้นเมื่อใด

ตอบ Algorithm A มี Worst Case Time Complexity เป็น $O(n^2)$ เพราะใช้ Nested Loop

ในการตรวจสอบอาคารทางด้านขวาของอาคารแต่ละหลัง Worst Case เกิดขึ้นเมื่อ

แต่ละอาคารต้องตรวจสอบอาคารทางด้านขวาหลายตำแหน่งก่อนจะพบ Next Greater Element หรือไม่พบเลย เช่น ข้อมูลเรียงจาก สูงไปต่ำ [5, 4, 3, 2, 1] ซึ่งทำให้ต้องตรวจสอบข้อมูลจำนวนมากที่สุด

4. Invariant ของ Monotonic Stack ในระหว่างการทำงานคืออะไร

ตอบ Invariant ของ Monotonic Stack คือ Stack ต้องรักษาลำดับความสูงของอาคารให้เป็น Monotonic Increasing โดย Index ของอาคารที่อยู่ใน Stack จะเรียงตามความสูงจาก น้อยไปมาก และเมื่อพบอาคารที่สูงกว่ายอด Stack จะทำการ Pop Index ที่พบ Next Greater Element ออก

5. จะขยายอัลกอริทึมนี้ไปแก้ปัญหา Largest Rectangle in Histogram ได้อย่างไร

ตอบ สามารถนำแนวคิด Monotonic Stack มาใช้โดยเก็บ Index ของแท่งอาคาร ที่มีความสูงเรียงจากน้อยไปมาก เมื่อพบแท่งอาคารที่มีความสูงน้อยกว่ายอด Stack ให้ Pop Index ออกมา แล้วคำนวณ พื้นที่ = ความสูง $\times$ ความกว้าง จากนั้นเก็บค่าพื้นที่ที่มากที่สุดจนประมวลผลครบทุกแท่ง ดังนั้นสามารถหา Largest Rectangle in Histogram ได้ด้วย Time Complexity $O(n)$

6. หากตึกเรียงจากสูงไปต่ำทั้งหมด Stack จะมีพฤติกรรมอย่างไร ต่างจากกรณีเรียงจากต่ำไปสูงหรือไม่

ตอบ หากตึกเรียงจาก สูงไปต่ำ ทั้งหมด Stack จะไม่เกิดการ Pop เนื่องจากตึกใหม่มีความสูงน้อยกว่ายอด Stack จึงทำการ Push Index ของตึกแต่ละหลังเข้า Stack ต่อเนื่อง ส่วนกรณีตึกเรียงจาก ต่ำไปสูง เมื่อพบตึกใหม่ที่สูงกว่ายอด Stack จะเกิดการ Pop Index ที่อยู่ใน Stack ออกก่อน แล้วจึง Push Index ของตึกใหม่เข้าไป ดังนั้น Stack มีพฤติกรรมแตกต่างกัน คือ สูงไปต่ำจะเน้นการ Push ส่วนต่ำไปสูงจะมีการ Pop ก่อน Push

7. Auxiliary Space ในกรณีเลวร้ายที่สุดของ Algorithm B คือเท่าใด และเกิดขึ้นเมื่อกรณีใด

ตอบ Auxiliary Space ในกรณีเลวร้ายที่สุดของ Algorithm B คือ $O(n)$ เพราะ Stack สามารถเก็บ Index ของอาคารได้มากถึง n ค่า กรณีที่เกิดขึ้นคือ ตึกเรียงจากสูงไปต่ำ เช่น [5, 4, 3, 2, 1] เนื่องจากไม่มีตึกที่สูงกว่ายอด Stack ทำให้ Index ถูกเก็บสะสมใน Stack และไม่มีการ Pop ออก